

ICT381 TMA — SkipQ (Group 5)

Cover (finalised by the student before submission)

- Course / assignment: ICT381 — DevOps assignment, TMA
- PI number: [PENDING — student input]
- Name: [PENDING — student input]
- Submission date: [PENDING — student input]
- Backend repository: <https://github.com/ArellaKoo/devops-assignment-BE>
- Frontend repository: <https://github.com/ArellaKoo/devops-assignment-FE>

Q1 — Capability audit and group provenance

The source is Group 5's GBA (T02): diner browsing/ordering/tracking (C1), vendor menu and order operation (C2), authentication and roles (C3), plus one extra story beyond the baseline. The audit in `docs/report/q1-audit.md` checks each capability against the GBA's user stories, acceptance criteria, feature/data/rule/workflow specification, class diagram and state machine, and persona screens — citing story and criterion identifiers.

All four statuses are partially supported: each original GBA artifact table has both useful evidence and additions or corrections. This assesses the GBA, not implementation completion. C1 delivers in-app Ready feedback rather than email notification and limits US6 to baseline status viewing (no preorder times, no registration); C3 replaces registration and user administration with the TMA's seeded-sign-in baseline. The audit records each correction with evidence: the GBA's "Completed" label corrected to Collected to agree with its own Q4/Q5; a 15-minute lock with 429 while locked added because the GBA's lock rule states no duration and no unlock mechanism; malformed ids in URLs forced to 404 rather than 500; and the conflicting GBA payment/accept/reject nodes settled into the one enforced lifecycle with its five permitted edges and the 30-minute no-show boundary.

Extra story (TMA Step 3). US10 — current and past orders — was chosen because it adds historical visibility using the orders the build already stores, and it reuses the Order List and Order Detail screens the baseline already needs: reuse instead of multi-stall payment or analytics complexity, and no reorder or analytics features the story does not ask for.

Provenance. Both repositories adapt the declared StaycationX lab baselines (commits in `docs/report/provenance.md`): the legacy Flask-MongoEngine adapter was replaced with direct MongoEngine, configuration externalised, CORS limited to the configured frontend, and `hotel/OneMap/session/Selenium/deployment` code excluded.

Q2 — Seed data and criterion coverage

The seed (`db_seed/seed.py`, data in `db_seed/fixtures.json`, run with `python -m db_seed.seed`) upserts by natural key, recomputes the time-relative fixtures on every run, and never deletes unrelated data.

`docs/report/seed-coverage.md` argues the coverage criterion by criterion rather than listing records: six users, two stalls, five menu items, two carts, nine orders. Each record exists to make a GBA criterion checkable: D1 owns orders in every lifecycle state plus a stale-snapshot order (O-C keeps an older name and unit price, which proves snapshots survive menu edits); D2's separate order (O-F) and the other stall's paid order (O-S2) expose ownership leaks and prove a closed stall keeps its queue; V1 and V1B share stall S1, proving vendor authorisation flows through the stall relationship; S2 is seeded closed while already carrying a paid order; M3 is seeded sold out and M4 soft-deleted; O-R and O-RO sit on opposite sides of the 30-minute no-show boundary relative to seed time.

Where a criterion describes an event or a refusal, the coverage table states that a seed record alone is not evidence and names the marker action that must produce it (the sold-out toast needs a second actor; the closed-stall checkout refusal needs a cart loaded before closure). States seeding alone cannot produce — transient invalid input, the five-failure lockout sequence, duplicate or concurrent checkout races, and the exact 30:00 instant — are listed explicitly, with the injected-clock unit tests and the functional races as their substitutes.

Q3 — Where role and scope enforcement live

All persona Blueprints share two helpers in `app/auth.py`: `current_user()` resolves the Bearer token into a trusted user — refusing 401 when the header is missing or malformed, the token was not signed with the configured secret, it is past its 3,600-second life (exactly 3,600 s still verifies), or its id matches no stored account — and `require_role()` raises 403 before any controller body runs when the role does not match. A role answer is not a scope answer: every scoped read and mutation in `app/models/` takes the actor and checks the relationship itself — `Vendor.authorizes`, `MenuItem.authorizes_vendor`, the cart owner check in `Cart.set_item`, `Order.get_for_diner/get_for_vendor/can_transition` — raising the same 403/409 codes whichever persona made the request; the actor always comes from the verified token and the record always from a model query scoped to that actor. The lifecycle guard lives in `Order.transition_to`, the single guarded path any caller must use: it re-checks role, stall ownership, current state, the five permitted edges and the 30-minute no-show boundary, then persists with an expected-current-state update so two racing vendors cannot both apply a move; the routes only translate the request and map the model's `DomainError` codes to the JSON error contract.

Q4 — Test suites: what the mocks can and cannot say

Q4(a) — which methods the offline tests actually exercise. The offline suite (306 tests, network-blocked by a socket sentinel) supplies records and stubs only the persistence boundary; the named methods run for real. `Order.transition_to` runs for real in `tests/unit/test_order_guard.py`: every one of the 36 state pairs, the 29:59/30:00/30:01 no-show boundary with an injected clock, the actor/scope refusals, and the expected-current-state update semantics (the race case records the exact update and filter, and the 0-modified lost race proves `invalid_transition` without moving the local document). `Order.place_from_cart` runs for real for the stored-key replay, cart revalidation, fingerprint comparison, snapshot construction and cart clearing; only `find_by_checkout_key`, `save` and the cart write are supplied. What the mocks cannot say is stated rather than claimed: asserting that a fake `list_for_diner` returns the rows the test told it to proves nothing about owner filtering, `status__in` semantics or newest-first ordering in MongoDB — filtering is driven through `GET /api/diner/orders?view=current|history` in the real-database suite against documents that only the database wrote, while ordering was observed in the browser checks; the checkout key's unique-index behaviour is proven only by the functional suite's concurrent checkout against the real index.

Q4(b) — two concrete strengths of the assessed suites. First, actual token/route/model integration rather than canned records: the functional tests never inject users — `User.create` writes a real hashed account, `POST /api/user/gettoken` must verify it and mint a signed 3,600-second token, and every subsequent request carries that token through `current_user()` → `require_role()` → the model scope check; unit tests supply stored accounts or query seams and cannot establish that complete persistence chain.

`test_diner_and_vendor_complete_order` walks Pending→Preparing→Ready→Collected with two separately signed tokens and asserts both the HTTP response and the diner-visible state after each vendor move — an end-to-end sequence the offline suites, which supply the actor directly, structurally cannot contain. Second, real database references, updates and fixture isolation: the two-thread same-key checkout hits the real (`diner`, `checkout_key`) unique index — exactly one request stores the order (201), the other returns the stored order as a replay (200), and `Order.objects(checkout_key=...).count() == 1` is read back from MongoDB, not from a mocked collection; the stale-checkout cases prove no order row was inserted for each refused key; the snapshot test re-reads the stored order after the menu item was renamed and re-priced. Fixture isolation is real: the conf test refuses the development database name before any connection,

registers every created document, sweeps route-created carts and orders by reference, and releases the MongoEngine alias after each app — which is why the suite was run twice against the same guarded `skipq_test` database (10 passed each time) with no manual reset between runs, leaving zero documents after each run.

Q4(c) — the designed extra-story suite (not implemented). US10's history cases are a design only: `docs/report/q4-extra-story-tests.md` carries the H01–H11 table in the worksheet's exact five columns (ID, short label, what passing proves, persona plus ordered endpoints, expected status plus state or message), each case naming the `view` parameter and the All/Current/Past UI mapping, with preconditions and the strategy/priority argument grounded in the augmented criteria — which cases the UI already exercises and which remain at design level. Nothing in the build implements that suite, and this report does not describe it as executed.

Q5 — Frontend architecture and flow evidence

The frontend is CRA with React Router 6; all routes live in `src/App.js` under a `RequireRole` gate and a persona layout that renders the shared feedback banner plus the outlet (route table in `docs/report/frontend-architecture.md`). The five shared decisions, each with the alternative it was weighed against and what it does not provide: (1) `src/config.js` — one configured API base URL matching the backend's CORS allowlist, so a fresh clone works from `.env.example`; no runtime discovery. (2) `src/api/client.js` — one fetch wrapper and one typed `ApiError` from the backend's stable error contract, native fetch rather than axios (its interceptor/retry features are unused), so token attachment and refusal parsing happen in exactly one place. (3) `FeedbackContext + FeedbackBanner` — a refusal raised on one screen survives a client-side redirect and lands where the user is; not a global toast system (no expiry timers, deduplication or translations). (4) `RequireRole` — a client-side gate that is explicitly not a security boundary: the backend re-verifies the token and role on every request (verified live: a diner's token was refused 403 on `GET /api/vendor/menu` while the UI kept running). (5) `AsyncButton/useAsyncAction` — a UI-side duplicate control for this tab's own clicks; a second tab, a second browser or a network-level resend is invisible to the button and is handled at the database layer instead, where the unique index makes a repeated checkout store at most one order, a matching replay returning the stored order and a conflicting reuse refused.

The flows: diner Login → Stalls → Menu → Cart → Checkout → Tracking, vendor Login → Menu → (item form) / Orders → Order detail. Sold-out and closed state are visible before the diner can act; the checkout key is minted when the page loads and retained across retries of the same attempt; the stale-cart refusal screen offers Review / Refresh-and-retry; and the vendor's order-detail controls render strictly from the response's `allowed_actions`, so the 30-minute no-show rule is visible in the UI, not only enforced by the backend. US10 reuses the Order List and Order Detail screens — no new history page: the list gains the All/Current/Past filter (Past never polls, because terminal orders cannot change), and the detail screen already renders terminal orders read-only with their notices. Flow evidence: `docs/report/q5-flows.md` (the two-context live verification paragraphs) and `docs/evidence/screenshots/` — `task8-01...18` (diner flow), `task9-01...27` (vendor flow), `task10-01...08` (US10 filters), `task11-01...11` (the two-persona lifecycle run).

Q6 — Browser-versus-functional result and load verdict

Q6(a) — the browser result the functional suite cannot establish. The Playwright suite signs the diner and the vendor in through the real login screen in two isolated browser contexts — no token is injected into either browser. The diner clears the seeded cart, adds one item and checks out through the screen that mints the checkout key itself; the paid screen displays the new queue number, and the vendor's polling queue surfaces that exact number before Accept, Mark ready and Mark collected on the vendor's screens. After each transition the diner's tracking screen — polling on its own timer in the other context — is observed to display Preparing, then the Ready-for-collection notice, then terminal Collected. An in-process functional test cannot establish this: the Flask test client does not render React routing or DOM banners, does not exercise the checkout key the UI mints, and cannot show two independently polling browsers converging on the persisted state. Run 1 passed in 12.02 s and run 2 in 11.48 s against the same seeded guarded database, no reset between runs (docs/evidence/browser-lifecycle/run-1.log, run-2.log; screenshots task11-01...11).

Q6(b) — endpoint, load and measurements. The endpoint loaded is the UI's 3-second menu poll, the most frequent diner request; order placement and checkout are once-per-order events and were deliberately not loaded. Locust ran at the polling cadence (2.9–3.1 s wait) with one sign-in per user plus a separate repeated-sign-in probe; main run: 10 users, 2 users/s, 120 s, on a local M3 MacBook Air (8 cores, 8 GiB, macOS 26.6.2), the Flask development server, MongoEngine 0.29.3 / PyMongo 4.18.2, MongoDB 8.0.32 on loopback, and the seeded dataset of 6 users / 2 vendors / 5 menu items / 9 orders, after a 5-user/60 s sanity run. Result: 395 menu requests, 0 failures — p50 5 ms, p95 20 ms, p99 65 ms, max 89.7 ms, 3.31 req/s; the 40 sign-ins were also 0 failures. Opt-in server-side instrumentation records that each menu request issues exactly three finds — users (1 row), vendors (1 row), menu_items (3 rows) — with combined driver-observed command duration p95 3.16 ms, API wall time p95 8.23 ms and full menu-list materialisation p95 2.01 ms (q6-performance/run-metadata.md).

Database verdict (≤ 100 words). At 10 users and 3.31 menu requests/s, 395 requests succeeded. Each issued three finds, including `MenuItem.list_for_vendor` returning three items. Combined driver-observed command duration was p95 3.16 ms; API wall time was p95 8.23 ms and Locust latency p95 20 ms. Menu materialisation was p95 2.01 ms. Command time represented a median 39% of paired API wall time. These measurements include transport and driver overhead; they do not isolate MongoDB execution or explain the remaining latency. No large query delay appeared on this small dataset; production bottlenecks remain inconclusive.

Q7 — Screencast and product hypotheses

Q7(a) — the recording. Preparation for the narrated ≤ 8 -minute 720p MP4 is in q7-screencast/ (backend repository): a timed narration script with the expected visible outcome stated before each step, the demo-data checklist, the recording steps, and a scripted demo run that executed the exact script headlessly and verified each visible refusal (q7-screencast/demo-run-log.md). The narrated recording itself is made by the student: [RECORDING LINK PENDING — human-only].

Q7(b) — two falsifiable hypotheses from the built MVP (400–500 words).

Diner hypothesis — stale-cart recovery. Pain: at checkout the server revalidates every line against the current records; if a line's price changed, an item went sold out, or the stall closed after the cart was loaded, the diner is refused on the stale-cart screen (docs/evidence/screenshots/task8-07-checkout-stale-refused.png) and must Review or Refresh-and-retry. Who experiences it: any diner whose menu changes while they browse — the refusal names the reason, not the offending line or its fix. Hypothesis: showing the specific stale line with an inline one-tap fix (remove it, or keep it at the new price) would measurably speed the diner from refusal back to a paid order; the observable benefit is fewer abandoned checkouts after a stale-cart refusal. Market and who pays: the same situation occurs wherever menus are live and shared — hawker centres, food courts, campus dining. As an explicit pilot assumption, ten stalls $\times$ 100 daily pickup orders gives 1,000 ordering occasions per day; if 10% become stale, 100 occasions could benefit. These are planning assumptions, not surveyed users or measured

demand. The stall loses the sale; a platform hosting the queue would pay to keep orders completing. Test: the cheapest evidence is a one-screen prototype of the inline fix shown to five to ten people after they have experienced a simulated stale-cart refusal; success is the majority completing the order without leaving the screen, falsification is most saying the plain message was enough or the inline fix adding no perceived speed.

Vendor hypothesis — no-show prediction. Pain: once an order is Ready, the stall has no signal that the diner is coming; after `ready_at` + 30 minutes Mark no-show becomes available alongside collection, where the payment stays paid but the prepared food is wasted (task9-12, task9-14, task9-27). Who experiences it: every stall, worst during the peak when a ready order idles. Hypothesis: a single diner-facing nudge about ten minutes after the order becomes Ready ("your order is ready — still coming?") would reduce the no-show rate; the observable benefit is fewer NoShow orders per day per stall. Market and who pays: SkipQ's 30-minute no-show rule defines this proposed pickup-stall pilot; a pilot assumption of ten stalls × 100 daily orders × 5% no-shows gives 50 affected orders per day and ten potential business customers. These counts are hypothetical; the seed proves the state exists, not its frequency. The stall pays through wasted food; a queue platform would pay to raise the completed-pickup rate. Test: the cheapest trial is a two-week instrumented A/B on one live stall — nudge on for alternate days, no-show rate compared against the stall's own baseline; success is a relative drop of at least 20% on nudge days, falsification is no difference, or complaint counts rising enough to outweigh the saved food cost.

Captions and evidence identification

The evidence this report cites, by file: `docs/report/q1-audit.md` — the four capability audits with their statuses and the maintenance log; `docs/report/seed-coverage.md` — the seed matrix and criterion argument; `docs/report/q3-roles.md` — role/scope placement and the live verification log; `docs/report/q4-suites.md` — the mock/real-DB boundary, the two suite strengths and the run results; `docs/report/q4-extra-story-tests.md` — the design-only H01–H11 cases; `docs/report/frontend-architecture.md` — the route table and the five decisions; `docs/report/q5-flows.md` — the flow descriptions, the two-context live verifications and the screenshot ledger (18 + 27 + 8 rows); `docs/report/q6-browser-lifecycle.md` — the Q6(a) finding, step record and run record; `q6-performance/` — the Locust CSVs/HTMLs, the instrumented JSONLs and `run-metadata.md`; `docs/evidence/browser-lifecycle/run-1.log` and `run-2.log` — the full pytest output of the two browser runs; `docs/evidence/screenshots/` — task8-* diner flow, task9-* vendor flow, task10-* US10 filters, task11-* the two-persona lifecycle.

Clarifications that keep the claims honest: the "5-second status-update target" is not claimed as measured — state changes were observed through polling with condition waits of up to 15 seconds, and no timing claim is made beyond the measured load figures above. Q4(c) is designed, not executed, and is labelled as such everywhere it appears. No frontend unit/component suite was created, because the TMA does not require one. The load verdict's dataset is the small seeded one, and the verdict says so. The AI-assistance disclosure with the exact prompts, question labels and verification notes is in `docs/assessment/ai-prompts.md`.

Appendix A — Q1(a) four capability audits

Group 5 (T02). This document is maintained during the build and will be finalised for the report. Each capability gets a Status followed by the five-row artifact table, citing the GBA by story/criterion/feature/screen name rather than transcribing it. Gaps discovered at any point are recorded in the affected row as though always known, with the evidence that settled them.

Status evaluates how completely the original GBA supports each selected capability. All four are partially supported because their artifact tables identify missing or conflicting evidence; implementing the additions does not change that original-evidence classification.

The GBA does not number its acceptance criteria; US7-AC2 means the second Given/When/Then block under User Story 7 in GBA Q2(b). The TMA's "Q2(c)" numbering refers to the same criteria; this report cites the story and block.

C1: Diner browsing, ordering, and tracking

Status: partially supported.

GBA artifact	Evidence it supports this capability	What needs adding/changing, and why
User stories: ID/text/persona	Diner US4 adds/changes cart items; US5 pays and receives a queue number; US6 views changing queue/order status. Their happy path covers ordering and tracking.	Limit US6 to baseline status viewing: the TMA requires simulated payment and the build uses in-app Ready feedback, so US5 is described as a simulation and email delivery is not claimed. Open-stall browsing needs the trading-state linkage from US12, which US6's screens do not show.
Acceptance criteria	US4-AC1/AC2 cover add/remove/quantity and the updated price; US5-AC2/AC3 distinguish paid confirmation with queue number from failure/retry; US2-AC4 blocks checkout of an item sold out after it was added; US6-AC1/AC3/AC4 cover refresh and terminal display.	Added outcomes the criteria never state: empty cart refused; positive-integer quantity/type rules with zero-removal; one-stall cart; server-derived cents total; a changed price/fingerprint requiring review (409 price_changed); sold-out/inactive/closed rechecks using current records at checkout; a failed payment creating no order/queue and preserving the cart; the same-key retry returning the original order while conflicting reuse is refused; and purchased name/price snapshots surviving menu edit/removal. The original criteria do not specify these refusals or duplicate handling, so each is asserted as an audited addition.
Specification: feature/data/rules/workflow	GBA Q4(a) "Add to cart" names Cart/Diner/item/quantity/unit price/total; "Payment" requires amount = total, no order or queue number before success, and success/failure retry; "Order status & notification" fixes the Pending→Preparing→Ready→Collected workflow.	Money is stored as integer cents and calculated server-side; the method and refusal codes, menu/stall revalidation, and the cart price fingerprint are specified by this build. Line-item purchase snapshots are persisted so history survives menu edits. In-app Ready feedback is the chosen notification channel; actual email is outside the selected scope.
Class diagram and state machine	GBA Q6(a) shows Diner→FoodOrder and FoodOrder→OrderedFood/Payment with FoodItem data; Q6(b) shows the payment/accept/preparation/collection concepts.	The diagram lacks the Cart/CartItem entities, the order-to-Vendor reference, the item image URL, the queue number, and the timestamp/price-snapshot fields; building app/models/ made each gap concrete. Q6's PendingPayment, Accepted, PendingPreparation, PendingCollection, Rejected, Stored and duplicate refund nodes conflict with the paid-before-order rule or duplicate the canonical states, so the build reconciles them to Pending, Preparing, Ready, Collected, Cancelled (payment Refunded) and NoShow (payment stays Paid), with Stored dropped as persistence rather than a workflow state.

GBA artifact	Evidence it supports this capability	What needs adding/changing, and why
Persona screens	GBA Q5's Diner Home / Vendor / Item Details / Cart / Checkout / Payment Success / Order Tracking screens cover the ordering sequence.	Preorder time and registration are removed from the selected scope. Added: open/closed/sold-out state visible before the action, explicit simulation success/failure with retry, pending-submit feedback, the stale-cart refusal screen, and a readable Ready banner. The Payment Success screen is merged into Order Tracking so the queue number and live status live on one screen; US10's history reuses the Order List/Detail screens (see C4).

C2: Vendor menu and order operation

Status: partially supported.

GBA artifact	Evidence it supports this capability	What needs adding/changing, and why
User stories: ID/text/persona	Vendor US1/US3 create/edit menus; US2 toggles sold out; US7 manages incoming paid orders; US12 opens/closes the stall.	US12 is High in GBA Q3's table but absent from GBA Q4's features and Q7's task list; the TMA baseline explicitly requires open/close, so it is included and the omission recorded here. US17 (capacity pause) is a different story and is not conflated with US12.
Acceptance criteria	US1/US3 validate required menu fields; US2-AC1/AC2 sold-out/restock; US7-AC1–AC6 incoming/accept/reject/ready/collect/no-show; US12-AC4/AC5 block new ordering while preserving existing orders.	US7-AC5's "Completed" is corrected to Collected to agree with GBA Q4/Q5; the disallowed-transition, vendor-ownership, and closed-stall existing-order outcomes are added, the no-show clock origin is defined at ready_at with the inclusive 30-minute boundary, and "accepting begins Preparing" is stated (US7-AC2 only says the status becomes Preparing).
Specification: feature/data/rules/workflow	GBA Q4(a) Menu Management fixes name 1–80, price >0/<9999 with two decimals, description ≤500, JPG/JPEG/PNG images; "Diner's order list" specifies the paid-only queue, rejection/refund and terminal immutability.	Added the explicit is_open trading state and open/close operations, paid-order scope restricted to the vendor's stall, the simulated refund on rejection, NoShow staying paid, soft-deletion for menu removal, the permitted next-action list, and the actionable 400/403/409 mapping with stable error codes.
Class diagram and state machine	GBA Q6(a) Vendor→Menu→FoodItem and FoodOrder/OrderedFood; Q6(b) shows rejection/refund and the 30-minute no-show concept.	The diagram does not distinguish a vendor account from the stall; the build models the stall (Vendor document) plus accounts that reference it, so several accounts can manage one stall. Added the Order→Vendor reference, the trading state, the ready_at timestamp, and the payment method/amount. The canonical reject is Order Cancelled plus Payment Refunded; Stored is persistence, not a lifecycle state, and the duplicate refund nodes are not two distinct states.
Persona screens	GBA Q5's Vendor Admin / Item Details / Diner's Order / Order Details screens include menu edits and the accept/reject/ready/collect/no-show controls.	Added the open/close switch and closed banner, current-state permitted controls (only the allowed next statuses are offered), the paid queue staying accessible after closure, and the refund/early-no-show/refused-transition displays. Business-insights and profile editing (US11) are outside the selected scope.

C3: Authentication and roles

Status: partially supported.

GBA artifact	Evidence it supports this capability	What needs adding/changing, and why
User stories: ID/text/persona	Vendor US8 signs in and edits only its own stall; Diner US16 signs in and accesses its own cart/orders.	Seeded account sign-in is the TMA baseline, so US16's registration side is adapted rather than built: the adaptation (seeded sign-in replaces registration/user administration, per TMA Q5) is stated instead of claiming full registration coverage.
Acceptance criteria	US8-AC1/AC2 correct/incorrect vendor login; US16-AC3/AC4 login outcomes and AC5 own-data visibility.	Added the outcomes the criteria omit: missing/invalid/expired token 401, wrong role 403, and foreign record 403 with no mutation or data leak; email normalisation; and the lock/unlock behaviour (failures 1–4 401, fifth failure locks 15 minutes and returns 429, attempts while locked 429, reset at expiry or on success). The GBA says "locked after 5 attempts" without a duration or unlock mechanism, so both are audited additions.
Specification: feature/data/rules/workflow	GBA Q4(a) User authentication: email/password/vendor_id, unique email, hashed credentials, correct-role redirect, lock after five failures; a vendor_id may have many users.	Specified signed expiring tokens (3,600 s) from the configured secret, model-owned scope checks, and a credential-free example config. The 15-minute lock and 429 while locked are set because the original lock has no unlock mechanism. The GBA's production HTTPS/uptime claims are not claimed by this locally hosted build.
Class diagram and state machine	GBA Q6(a) abstract User with email/password and Diner/Vendor specialisation.	Store the password hash rather than the raw password; model one User document with a role plus optional Vendor reference so several accounts can share a stall, and add the failure-count/locked-until fields. Authentication events are not fulfillment states.
Persona screens	GBA Q5's shared Login and Diner-only Registration screens distinguish personas.	Keep the seeded Login only; add the role redirect, protected navigation, shared token-expiry feedback, and logout. UI route gating is usability; the server remains the security boundary.

C4: Extra story US10 — current and past orders

Status: partially supported.

Reason for choice (TMA Step 3): US10 is a Medium-priority GBA Q3 backlog story outside the baseline order-status tracking. It adds historical visibility using the orders the build already stores and the Order List and Order Detail screens the baseline already needs, so it reuses functionality instead of adding multi-stall payment or analytics complexity. It was chosen as the one story beyond the baseline, and its selection is approved for this build.

GBA artifact	Evidence it supports this capability	What needs adding/changing, and why
User story: ID/text/persona	US10: "As a diner, I want to be able to view my past orders, so that I can know what I have ordered previously." US16-AC5 supplies the ownership intent (only that diner's information is displayed).	Treat past-order visibility as the extra capability, distinct from the mandatory tracking of a new order (US6). Reorder and analytics are not requested by the story and are not added.
Acceptance criteria	US10-AC1 says the Order List page shows all current and past orders when scrolled.	Added the outcomes the single criterion cannot determine: All default plus Current/Past filters, empty results, only-own orders, newest-first sort with an equal-time tie-breaker, read-only historical detail, and unchanged purchase snapshots. Each is asserted as an audited addition.

GBA artifact	Evidence it supports this capability	What needs adding/changing, and why
Specification: feature/data/rules/workflow	GBA Q4(a) authentication mentions individual history; the order features list Order ID/items/quantity/status/payment, but Q4 has no history feature.	Added the explicit query contract: <code>view=all current history</code> (invalid view 400), <code>Past = Collected/Cancelled/NoShow</code> , empty list 200 with <code>items: []</code> , owner filtering inside the model, and newest-first by <code>created_at</code> with id tie-breaker. The immutable name/unit-price snapshots make history trustworthy after menu edits or removals.
Class diagram and state machine	GBA Q6(a) Diner creates zero/many FoodOrders with date/time, cost and OrderedFood quantity/subtotal; Q6(b) mentions terminal orders and Stored.	Orders are kept in the database after <code>Collected/Cancelled/NoShow</code> without a <code>Stored</code> workflow state or a yearly deletion job (retention scope: the local build keeps them). Added the immutable purchased name/unit-price snapshots and the own-diner reference the diagram lacks.
Persona screens	GBA Q5's Order List Page lists current and past orders with statuses.	One list with <code>All/Current/Past</code> and a readable empty state; the Order Detail screen is reused for read-only past details, so no separate history page is built. Multi-stall cancellation actions are removed because multi-stall and diner-cancel-anytime are out of scope and conflict with the vendor's preparation rules.

Audit maintenance log

Date	Capability	Discovered gap	Decision and evidence
2026-10-07	C1/C2	GBA Q6 class diagram has no Cart/CartItem, queue number, image URL, trading state, order→vendor link, or price snapshots	Modelled each explicitly in <code>app/models/</code> ; seed order O-C keeps an older name/price snapshot to prove the point (see <code>seed-coverage.md</code>).
2026-10-07	C2	US12 open/close is High in GBA Q3 but missing from GBA Q4/Q7	Included in scope per TMA baseline; recorded here and in the C2 story row.
2026-10-07	C2	US7-AC5 says "Completed" while Q4/Q5 use Collected	Canonical terminal state is Collected; recorded in the C2 criteria row.
2026-10-07	C3	GBA lock rule has no duration or unlock mechanism	15-minute lock, 429 while locked, reset at expiry/success; implemented in <code>User.authenticate</code> and unit-tested with an injected clock.
2026-10-07	C4	US10's single criterion cannot determine filters/sorting/ownership outcomes	Added All/Current/Past contract, empty/sort/ownership outcomes; designed (not implemented) as Q4(c) cases.
2026-10-07	C3	Token lifetime needs an explicit boundary rule and live proof	<code>app/auth.py verify_token</code> accepts exactly 3,600 s (inclusive) and refuses 3,601 s/future-dated tokens (unit, injected clock). Live <code>POST /api/user/gettoken</code> verified 200/400/401/429 on the dev API, including the 5-attempt lockout on a throwaway account; protected-route 401/403 over HTTP verified live in Task 4 (missing/garbage/expired token 401, wrong persona 403). See <code>docs/report/q3-roles.md</code> .
2026-10-07	C1/C2	Malformed ids in URLs must be 404s, not 500s, and the model <code>get()</code> seems misused <code>MongoEngine's with_id</code> (which returns the document, not a queryset)	<code>Vendor.getItem.getOrder.get</code> now treat malformed ids as missing (unit + live 404 checks); patching/deleting a cart line that is not present returns 404, and closed stalls stay reachable by direct link while new cart adds are refused 409 (unit + live two-persona checks, 7 October).
2026-10-07	C1/C2	The GBA's conflicting payment/accept/reject nodes must be settled into one enforced lifecycle, and the C2 "closed stall keeps its paid queue" row needed live proof	<code>Order.transition_to</code> is the single guarded path: five permitted edges, 30-minute NoShow boundary, expected-current-state persistence, cancellation embedding Refunded, NoShow staying Paid; 75 offline guard cases, 38 offline route cases, and the live dev-API walk (including the paid queue moving while the stall was closed) confirm it — see <code>docs/report/q3-roles.md</code> (Task 5, 7 October).

Date	Capability	Discovered gap	Decision and evidence
2026-10-07	C1	The diner screens (stalls/menu/cart/checkout/tracking) had to exist for the audited C1 outcomes to be observable, and the US10 filter row was still open	Full diner screens landed with Task 8: sold-out/closed state visible before the action, add-increment semantics, cart stepper/removal with the server total, the retained-key checkout with the stale-cart refusal screen, Ready banner and terminal notices, 3 s polling with cleanup; 21-check live run ×2 and the 18-screenshot ledger in docs/report/q5-flows.md. Remaining C1/C2 screen gaps: the vendor screens (Task 9) and the Order List All/Current/Past filter (Task 10).
2026-10-07	C2	The vendor screens did not exist yet, so the audited C2 outcomes (open/close control, sold-out toggles, item add/edit/remove, the paid queue, the allowed-controls-only rule, the 30-minute NoShow boundary, terminal read-only state, refund/no-show displays, closure-preserves-queue) had no observable surface	Full vendor screens landed with Task 9: the trading-state switch and closure banner on the menu screen, per-item sold-out toggles and confirm-gated removal, the create/edit form that shows the backend's own 400 messages, the paid queue (newest-first, polling while any order is active, fully accessible while the stall is closed) and the detail whose controls are rendered strictly from <code>allowed_actions</code> — Pending offers only Accept/Reject, Preparing only Mark ready, Ready gains Mark no-show only past <code>ready_at+30 min</code> , and terminal orders show no controls; the 37-check two-context live run ×2 (diner places, vendor fulfils; the seeded Cancelled/NoShow/stale-snapshot cases shown on both personas) and the 27-screenshot ledger in docs/report/q5-flows.md confirm the row, and C2 is implemented; its original GBA evidence remains partially supported.

Date	Capability	Discovered gap	Decision and evidence
2026-10-07	C4	The US10 All/Current/Past screens did not exist, so the audited additions (filter membership, empty results, newest-first ordering, read-only past detail, snapshot stability, per-view empty copy) had no observable surface, and Q4(c)'s designed H01–H11 cases still had no screens to aim at	Task 10 landed the filter group on the existing Order List (All default; Current/Past map to the server's view filters; Past never polls while All/Current keep the 3 s polling; per-view empty states) and reused the Order Detail screen read-only for past orders — no separate history page; the 15-check two-context live run ×2 (mixed-status diner across both stalls, another diner's order excluded, an order-free diner's three empty states, a live reprice leaving the purchase snapshot intact, windowed request counts proving the polling split) and the 8-screenshot ledger in docs/report/q5-flows.md confirm the row; the designed cases in docs/report/q4-extra-story-tests.md stay unimplemented per the TMA.
2026-10-07	C1–C4	Final reconciliation against the finished code: capability status assesses the original GBA, not whether implementation is finished	Verified against the final suites and the final app: the offline unit suite is 302 cases (75 lifecycle guard cases in test_order_guard.py, 38 order-route cases in test_order_routes.py, plus cart/validation/auth/foundation/query-timing), the functional suite passes twice against the guarded database, the Playwright suite passes twice with UI logins for both personas, and the Q7 demo script executed the full two-persona walk plus both refusals headlessly (q7-screencast/demo-run-log.md). Task 12's opt-in query/request timing is a no-op unless SKIPQ_QUERY_TIMING=1 is set on the server process, so it changes none of the audited behaviour or status codes; C1 and C3 therefore remain partially supported (US5 payment is simulated and US6's email channel is out of scope; US16's registration side is replaced by seeded sign-in per the TMA), while C2 and C4 also remain partially supported.

Appendix B — Q1(b) module provenance

This records the declared lab reuse and the final distinction between copied scaffolding and newly implemented SkipQ logic. Original source commits were recorded during adaptation; the reference clones are no longer present in this workspace at final inspection.

Backend source: [ArellaKoo/StaycationX_Backend](#), commit 153ce9004ceb6fde84c35873b53b4957c76062a3.

Freshness check after the user synced the lab folders: backend main is now

7d509dbcdeac18020a47677b79107805cd0f31cb (2 October 2026). The only change from the recorded adaptation baseline is DockerfileMongo: FROM mongo becomes FROM mongo:7, plus a final newline. Application modules, requirements, tests, README and seed files are identical. Docker is outside this local TMA build, so no new application adaptation is required. That was the source snapshot checked during adaptation.

Frontend source: [ArellaKoo/StaycationX_Frontend](#), commit bc3367629ff31b920634dc3c046684f8c3877059. Its manifest and file structure identify the lab React application described in the assignment as myReactApp.

The frontend source snapshot checked during adaptation matched that commit. Assignment implementation is in backend/ and frontend/.

Source module	Actual reuse/adaptation	What it does and why SkipQ needs it
Backend app/__init__.py	Factory structure substantially rewritten in the same target path; no hardcoded lab secrets or hotel Blueprint imports copied	create_app assembles a configured Flask application. Development and tests configure their own database in separate processes; fixtures explicitly release the global MongoEngine alias before creating a differently configured app.
Backend app/extensions.py	Separate initialization pattern retained; rewritten for direct MongoEngine and a configured CORS origin	Database and browser-origin setup remain separate from routes. Direct MongoEngine avoids the old Flask-MongoEngine JSON encoder adapter while retaining the required document model approach.
Backend requirements.txt	Dependencies reviewed and reduced to the REST stack, with versions pinned after installation	Replaces legacy Flask/Werkzeug/form/Selenium dependencies with the libraries this local REST application needs. This is adapted dependency scaffolding, not original SkipQ domain logic.
Frontend public/index.html	Copied shell, adapted metadata/title and removed unused template assets	Provides the one HTML document mounting the React SPA.
Frontend src/index.js	React 18 createRoot entry pattern retained; removed unused web-vitals instrumentation	Starts the React application and supports client-rendered screens.
Frontend src/index.css	Base font/smoothing styles copied	Provides readable system fonts without extra styling scope.
Frontend src/App.js	BrowserRouter/Routes/Route pattern substantially rewritten; hotel/OneMap imports and routes omitted	Defines the final nested diner/vendor routes and their shared role gates/layouts.
Frontend package.json	CRA/React/React Router/Bootstrap basis adapted; regenerated lockfile; react-scripts updated to 5.0.1 and compatible TypeScript 4.9.5 build peer pinned	Keeps the lab build approach while resolving the manifest/lock mismatch, Webpack 4/OpenSSL failure and incompatible optional peer observed on Node 22. Scaffold generator and unused frontend test packages/scripts were removed; JavaScript app code remains JavaScript.

No hotel models, booking/package endpoints, lab seed BSON/users CSV, hardcoded keys, OneMap components, Selenium tests, Docker images or deployment files are imported. Their absence is intentional adaptation to the TMA scope.

The lab factory, extension, React-entry and routing scaffolding above is the declared reuse. SkipQ domain models (app/models/), signed-token/role enforcement (app/auth.py), REST persona controllers, fixture seed data and assessed tests are new application-specific implementations, not copied hotel models or test files. The lab's legacy hashing/session approach was not retained; credentials use the current Werkzeug hash API

and tokens use `itsdangerous`. Frontend persona pages, the shared API client, authentication/feedback contexts and mutation guards are new SkipQ code within the adapted CRA/router structure.

AI assistance is recorded separately in `docs/assessment/ai-prompts.md`; the student must retain exact prompts and verify/explain submitted code.

Foundation error handling follows the response-preserving pattern in the [Flask error-handling documentation](#). Explicit fixture teardown follows [MongoEngine's documented connection lifecycle](#). Startup regression cases prove these two integration assumptions; they are not the assignment's domain/lifecycle coverage.

Appendix C — Q2(c) criterion-by-criterion seed coverage

Status: implemented and verified. The seed script is `db_seed/seed.py` (data in `db_seed/fixtures.json`, run with `python -m db_seed.seed`), and the seeded credentials are documented in `README.md`. This document argues the seed's coverage criterion by criterion, as the report needs.

Seed records and what they establish

All records use deterministic handles; re-running the script upserts the same records and refreshes time-relative fixtures. It never deletes unrelated data.

Handle	Record and why it is needed
D1 / D2 / D0	<code>diner.one@skipq.test</code> , <code>diner.two@skipq.test</code> , <code>diner.empty@skipq.test</code> . D1 owns current and past orders across every lifecycle state; D2 owns foreign orders that expose ownership leaks; D0 has no orders for the empty-state requirement.
V1 / V1B / V2	<code>vendor.one@skipq.test</code> and <code>vendor.one.backup@skipq.test</code> both reference stall S1; <code>vendor.two@skipq.test</code> references S2. Two accounts on one stall test that vendor authorisation flows through the stall relationship, not one user. All login counters start at zero.
S1 / S2	S1 is open; S2 is closed and already carries a paid order. Closed stalls stay reachable through their direct menu link and must display closed state, while open-stall discovery excludes them.
M1 / M2 / M3 / M4	S1 has two available items priced 6.50 and 2.50, one sold-out item, and one soft-deleted item. Each has a name, description and bundled image served at a valid PNG URL. Distinct prices make arithmetic errors visible.
M5	Available S2 item, so opening S2 enables a cross-stall cart attempt.
C1	D1's cart: two M1 and one M2, expected total 1,550 cents. D2 has a distinct cart on S2. D0 has no cart.
O-P, O-PR, O-R, O-R0	D1/S1 paid orders in Pending, Preparing, recently Ready, and Ready at least 30 minutes. Recent/overdue timestamps are computed relative to seed time, so the two Ready records are always on opposite sides of the no-show boundary at verification time.
O-C, O-X, O-N	D1 terminal orders: Collected/Paid, Cancelled/Refunded, NoShow/Paid. O-C's line keeps an older name and unit price than the current menu item, which proves snapshots survive menu edits. Distinct creation times and queue numbers establish history and sorting.
O-F, O-S2	D2's distinct historical order (isolates diner ownership) and an existing S2 paid Pending order (shows closing a stall does not erase its queue).

Criterion-by-criterion argument

Shared criteria are listed once with both capabilities named. Where a criterion describes an event or a refusal, a seed record alone is not evidence that the notification, guard, or validation works, so the marker action is stated.

Capability / GBA criterion	Seeded precondition and marker action	Expected result and why the state is sufficient
C1/C2 US1 AC1: add menu item	V1 signs in to S1; submits a new valid item.	201; reload gives 200 and the saved fields. A vendor/stall reference must exist, but a pre-existing item would not prove creation.
C2 US1 AC2: missing required fields	V1/S1; submit an item without a name, then without a price.	400 with a field-specific error and no new item. Invalid records are produced by the action, not seeded.
C1/C2 US1 AC3: display menu details	D1 selects open S1 with M1/M2.	200 menu; images, prices and descriptions render in the browser. API data alone does not prove rendering.

Capability / GBA criterion	Seeded precondition and marker action	Expected result and why the state is sufficient
C1/C2 US2 AC1: mark sold out	V1 updates available M1.	200, availability false; a fresh read persists it.
C1/C2 US2 AC2: restock	V1 updates sold-out M3.	200, availability true; a fresh read persists it.
C1/C2 US2 AC3: toast on availability change	D1 stays on S1's menu; V1 marks an initially available item sold out.	The newly observed change produces the toast and greyed-out item. Seeded M3 proves initial display only; the cross-context action produces the notification event.
C1/C2 US2 AC4: sold-out checkout	D1 loads checkout from C1; V1 then marks M1 sold out.	Checkout 409 names M1; the cart remains and no new paid order or queue number exists. Loading the checkout first makes this a server stale-data guard test.
C2 US3 AC1: valid edit	V1 edits M1's name, price, description and image.	200; fresh vendor/diner reads show the saved fields.
C2 US3 AC2: invalid edit	V1 submits a missing name or invalid price/image.	400; the error leaves the previous saved item unchanged. No invalid stored fixture is needed.
C1 US4 AC1: add/remove	D1 starts from C1 or clears it, then adds/removes available M1/M2.	New line 201, read/remove 200; lines and total match the remaining quantities.
C1 US4 AC2: change quantity	D1 changes M1's quantity in C1 from two to three.	200; total becomes 2,200 cents. Updating to zero removes the line; invalid quantities return 400.
C1 US5 AC1: preferred payment method	D1 has available items on open S1; repeat with a fresh order per method.	Valid simulated PayNow/Card/Cashless checkout is 201; recorded method and amount match the order. No gateway or financial details are used.
C1 US5 AC2: success and queue number	Same precondition; choose the successful simulation.	201 with a Pending/Paid order, queue number and payment-success screen. No pre-seeded paid order can prove this event.
C1 US5 AC3: failure and retry	Same precondition; choose the failed simulation, then a successful retry.	Failure 409 with no order/queue and an unchanged cart; the subsequent valid success is 201. The failure message and retry control are checked in the browser.
C1 US6 AC1: latest status on refresh	D1 opens O-P; V1 accepts; D1 refreshes.	Status read 200 shows Preparing in the API and on screen. Two actors create the change the refresh must fetch.
C1 US6 AC2: Ready notification	V1 advances O-PR to Ready while D1 tracks it.	Update/read 200 shows Ready plus in-app ready feedback. GBA email delivery is an explicitly narrowed scope item; no email evidence is claimed.
C1 US6 AC3: not-ready refresh	D1 refreshes O-P or O-PR.	200 shows the actual Pending/Preparing state, queue number, and no Ready indication.
C1 US6 AC4: collected refresh	D1 reads O-C, and separately refreshes a newly collected order.	200 shows Collected. The seeded record proves display; the transition action proves refresh sees a change.
C2 US7 AC1: incoming paid order	V1 reads O-P, then refreshes after D1 places a fresh order.	200 list/detail contains the order, queue number, item names and quantities; the new order appears. Failed payments never enter this list.
C2 US7 AC2: accept	V1 accepts O-P.	200, Preparing.
C2 US7 AC3: reject/refund	V1 rejects O-P or a fresh S1 Pending order; alternatively V2 rejects its own O-S2.	200, Cancelled with payment Refunded; D1's read agrees. The refund is simulated.
C2 US7 AC4: ready	V1 marks O-PR Ready.	200 with ready_at; D1 sees ready feedback.
C2 US7 AC5: collected	V1 collects O-R.	200, Collected. The audit replaces this criterion's inconsistent "Completed" label with Collected, matching GBA Q4/Q5.

Capability / GBA criterion	Seeded precondition and marker action	Expected result and why the state is sufficient
C2 US7 AC6: NoShow	V1 acts on O-RO and tries the early action on O-R.	Overdue 200 NoShow/Paid; premature 409 with Ready unchanged. The exact 30-minute boundary uses an injected clock in unit tests rather than waiting.
C3 US8 AC1: vendor valid login	Sign in with V1's credentials; repeat with V1B.	Token 200; the UI opens the vendor's own menu screen and displays S1. V1B separately confirms shared-stall access.
C3 US8 AC2: vendor invalid login	Submit a wrong password for V1.	401 with a helpful message and retry option.
C2 US12 AC1: open stall	V2 opens S2.	Update 200, is_open true; discovery includes S2 and a valid checkout can create an order.
C2 US12 AC2: close stall	V1 closes S1.	Update 200, is_open false; new checkout is refused 409.
C1/C2 US12 AC3: show closed	D1 visits S2 through its direct menu link.	Menu 200 carries closed state; the browser shows Closed and open discovery excludes the stall.
C1/C2 US12 AC4: prevent closed checkout	D1 loads a valid S1 checkout before V1 closes S1.	409 closed-stall message; no new order or queue, cart retained. S2's seeded-closed state also proves a direct attempt is refused.
C2 US12 AC5: preserve paid queue	V2 reads O-S2 while S2 is closed, then processes it.	List 200 still includes it; valid lifecycle updates remain 200.
C3 US16 AC3: diner valid login	Sign in with D1's credentials.	Token 200; the UI opens the diner stall list.
C3 US16 AC4: diner invalid login	Submit a wrong password for D1.	401; access denied with a useful error.
C3 US16 AC5: own cart/orders/history	D1/D2 have distinct carts and orders; sign in separately, then request the other diner's existing order.	Lists/cart 200 include only own records; a direct foreign order read is 403. Authentication and owner filters are both necessary.
C4 US10 AC1: all current/past own orders	D1 reads view=all, then compares the current/history filters.	200; All includes O-P/O-PR/O-R/O-RO/O-S2 and O-C/O-X/O-N, excludes O-F; current/history partition it. The browser reaches the oldest order.

States seeding alone cannot produce

- Transient invalid input (bad price strings, over-long names, malformed quantities, non-image URLs): produced by submitting the invalid value.
- Lockout sequence: five consecutive failed logins; the seed resets all counters to zero on every run so the marker can start clean.
- Duplicate/concurrent checkout races: require two submissions of the same checkout key; the functional and unit suites cover them.
- Notification events (sold-out toast, Ready banner): require a second actor to change state while the first persona stays on screen.
- The exact 30-minute boundary: O-R/O-RO sit on opposite sides relative to seed time, but the precise 29:59/30:00/30:01 instants are unit-tested with an injected clock.

Notes for the audit

- US16 AC1/AC2 (registration) is outside this TMA's seeded-login baseline and is recorded as a C3 scope adaptation in Q1, not as covered seed data.

- Email delivery and real payment processing are not seedable evidence of this build; both are explicitly narrowed in Q1.
- The GBA's PaymentMethod is represented by the typed payment-method choice on the embedded Payment; saved payment instruments are out of scope.

Appendix D — Q4(c) designed cases, strategy and priority

Status: designed, not implemented. The TMA asks for these extra-story cases to be designed and prioritised, not written or run; this table and the two paragraphs below are the deliverable. The US10 screens themselves (All/Current/Past on the existing Order List, read-only past detail on the existing Order Detail screen) are implemented and were verified live in Task 10 — that verification is ordinary task evidence, not this designed suite.

Preconditions and conventions. Preconditions use the demo handles or equivalent isolated real-database fixtures; each persona obtains a token through `POST /api/user/gettoken (200)` unless the authentication failure is the point of the case. `view=all|current|history` maps to the UI's All/Current/Past filters. The original US10 scroll requirement needs browser evidence as well as an API list (H11).

Test case ID	Test case - a short label	What passing it would prove	Persona, and the sequence of endpoints it calls	Expected outcome - the status code, and the state or message the response carries
H01	All own orders	Original US10 AC covers both current and past orders, including every supported state, without another diner's orders.	D1 login → GET <code>/api/diner/orders?s?view=all</code> .	200; own Pending/Preparing/Ready/Collected/Cancelled/NoShow records present; O-F absent; IDs unique, queue numbers and statuses present.
H02	Current/Past partition	Filtered views account for all orders without losing or duplicating one between groups.	D1 login → GET same list with <code>view=all</code> , <code>view=current</code> , then <code>view=history</code> .	All 200; Current only Pending/Preparing/Ready; Past only Collected/Cancelled/NoShow; sets disjoint and their union equals All.
H03	Purchase snapshots survive edits	Historical item names, unit prices, quantities, totals and payment details reflect the purchase after vendor rename/reprice/remove.	D1 login → GET historical O-C detail; V1 login → PATCH linked <code>/api/vendor/menu/<id></code> → DELETE same item; D1 GET O-C detail again.	Reads/updates/soft delete all 200; historical snapshot and total/payment/queue unchanged; purchase remains readable after soft deletion.
H04	Another diner's history	Both collection queries and direct detail access enforce ownership.	D1/D2 log in separately → each GET <code>view=all/history</code> → D1 GET <code>/api/diner/orders/<O-F-id></code> .	Lists 200 contain only actor-owned records; foreign existing detail 403, useful authorization error and no order details.
H05	Authentication and role	History cannot be accessed anonymously, with invalid/expired tokens, or by a vendor persona.	GET list and own detail with no/invalid/expired token; V1 login → same GETs.	Each absent/invalid/expired token request 401; each vendor request 403. No orders or payment fields leak in errors.
H06	Empty history	A legitimate diner with no prior orders receives an ordinary empty collection.	D0 login → GET <code>view=all</code> , <code>current</code> , <code>history</code> .	All 200, items: []. Friendly empty-screen copy needs the browser check in H11; the API cannot prove it.
H07	Newest first	Order lists follow the audited newest-first rule and remain stable across repeated reads.	D1 login → GET <code>view=all</code> and <code>history</code> , repeat each.	200, descending <code>created_at</code> , then ID descending for equal-time ties per the proposed design; identical order between repeated unchanged reads.
H08	Completed order becomes past	A newly collected own order remains available and moves from Current to Past.	D1 login/add cart/create order; V1 login → PATCH Preparing → Ready → Collected; D1 GET <code>all/current/history</code> and the captured detail.	Cart create/order create 201, transitions/list/detail 200; new order in All/Past, absent Current, detail Collected with original snapshots. This remains a designed case, not an extension to Q4(b)'s implemented suite.

Test case ID	Test case - a short label	What passing it would prove	Persona, and the sequence of endpoints it calls	Expected outcome - the status code, and the state or message the response carries
H09	Bad view or absent record	Invalid list input and a genuinely absent order have distinguishable JSON outcomes.	D1 login → GET view=unexpected → GET detail with a well-formed nonexistent ID.	Invalid view 400, validation message; absent detail 404, missing-order message. Foreign existing IDs remain H04's 403.
H10	Read-only past detail	Diner cannot mutate a historical order through vendor controls/endpoints.	D1 login → GET O-C detail → PATCH /api/vendor/orders/<O-C-id>/status; V1 login → PATCH same terminal order to Preparing; D1 rereads detail.	Detail 200; diner PATCH 403; vendor invalid terminal transition 409; reread 200, Collected/snapshots unchanged. UI absence of edit controls cannot be proved through API alone.
H11	Scroll and navigation	Original story's oldest entry is actually reachable, and detail/empty states work on the shared Order List/Detail screens.	Not fully runnable against API. D1/D0 login and GET list/detail establish data 200; separately use browser/manual navigation to /diner/orders, choose All/Past, scroll to oldest record and open it.	API 200 has complete own list; browser shows all own current/past entries, visible queue/status, oldest row, correct read-only detail and friendly empty state. No separate history page is required.

Strategy

US10 contributes one original acceptance criterion — the Order List page shows all current and past orders when scrolled. The suite starts from that criterion and covers its data completeness (H01: every supported state, and no other diner's orders), both halves of its partition (H02: Current and Past together account for the All set without overlap or loss), actor ownership (H04: the list queries and the direct detail read both refuse a foreign diner) and the scrolling interaction itself (H11: the oldest entry reachable in the real screen, with the friendly empty states). The single criterion cannot decide the edges, so each audited addition declared in Q1 gets its own case: H03 and H10 for the immutable purchase snapshots and read-only past detail, H05 for the token/role boundary around the whole feature, H06 for the empty collection, H07 for the newest-first ordering with its equal-time tie-break, H08 for the lifecycle transition that moves a live order into Past, and H09 for the two distinct error outcomes (invalid view versus absent record). H01/H02 and H11 answer the original criterion; H03–H10 answer the declared audited additions. Together they support a coverage claim for the declared story and its additions — not a claim that every conceivable defect in the ordering feature has been tested.

Priority

Write H01 first, because it establishes the story's central value (one list, both halves, own orders only) and every later case reuses its setup. Next H04 and H05, because a history feature that leaks another diner's purchases or serves orders without a valid token is the most serious failure this feature can have; security and privacy outrank presentation. Then H03, because the purchase history must be truthful after the vendor edits or removes the underlying menu items — the snapshot rule is the story's reason to exist. H02 and H08 follow: the filter partition must be exact, and an order that just finished must actually move into Past (lifecycle persistence). H06, H07 and H09 cover the routine behaviours (empty result, stable newest-first ordering, distinguishable error outcomes), and H10 confirms the read-only guarantee from the diner's side once the read paths are in place. H11 last: it exercises the remaining UI-only claims (scrolling to the oldest row, visible queue numbers and statuses, the read-only detail, the friendly empty screens) and can only be meaningful after the screens exist. In short: security and the preservation of purchase facts come before presentation refinements, and the browser-only evidence comes after the API-level cases. This is a hypothetical implementation order only — Q4(c) requests the design, not the implementation.

Appendix E — Q5(a) routes and shared decisions

Status: shared modules, routes and verification landed in the frontend repository; this note is condensed into the report at Task 13.

Route table

All routes are defined in `src/App.js` (CRA + React Router 6). The diner and vendor areas are nested under a `RequireRole` gate and a persona layout that renders the shared feedback banner plus `<Outlet/>`. No registration, user-administration or OneMap area exists, and no order screen is public.

Route	Screen	Access
<code>/login</code>	LoginPage — seeded-account sign-in with the backend's refusal text	public
<code>/diner/stalls</code>	StallsPage — open-stall discovery	diner
<code>/diner/stalls/:stallId</code>	MenuPage — one stall's menu	diner
<code>/diner/cart</code>	CartPage — cart, server total, fingerprint	diner
<code>/diner/checkout</code>	CheckoutPage — simulated payment, retry key and visible refusals	diner
<code>/diner/orders</code>	OrdersPage — own order list	diner
<code>/diner/orders/:orderId</code>	OrderDetailPage — own order detail	diner
<code>/vendor/menu</code>	VendorMenuPage — own menu + trading state	vendor
<code>/vendor/menu/new</code>	MenuItemFormPage — validated item creation/editing	vendor
<code>/vendor/menu/:itemId/edit</code>	MenuItemFormPage — validated item creation/editing	vendor
<code>/vendor/orders</code>	VendorOrdersPage — stall's paid queue	vendor
<code>/vendor/orders/:orderId</code>	VendorOrderDetailPage — one paid order	vendor
<code>*</code>	NotFoundPage — reachable catch-all	public

Tasks 7–10 implemented the shared modules and all diner/vendor screens. The current Order List includes All/Current/Past views; checkout and item forms are complete. Accessible headings, links and labels support the browser lifecycle suite.

Five shared decisions

Decision	Where it lives — module/component	Why there, against the alternative	What it does not give you
Resolving the API's base URL	<code>src/config.js</code> exports <code>API_BASE_URL</code> from <code>process.env.REACT_APP_API_BASE_URL</code> , documented in <code>.env.example</code> as <code>http://127.0.0.1:5001</code> .	One value every page and the shared client read; the documented default matches the backend's CORS allowlist for <code>http://127.0.0.1:5173</code> , so a fresh clone works with <code>cp .env.example .env</code> . Deriving the origin from the page would break the moment the frontend is served from a different port than the CORS allowlist expects. Alternative: Reading <code>process.env</code> in every page, or deriving the origin from <code>window.location</code> .	No runtime discovery: a backend moved to another host requires the <code>.env</code> change; the file holds no other configuration and no per-route overrides.
Attaching the token to a request	<code>request(path, options)</code> attaches the Bearer token, resolves the base URL, and throws <code>ApiError(status, code, message, plus expired for 401 authentication required)</code> parsed from the backend's stable <code>{"error": {"code", "message"}}</code> contract.	Token attachment and refusal parsing happen in exactly one place, so no page can forget the header or render a raw rejection; native <code>fetch</code> plus a small error class covers the whole contract without a new dependency. Axios was a real alternative (interceptors, retries) but its features are unused: the contract is one header and one error shape, and adding a library would obscure where the token leaves the browser. Alternative: Raw <code>fetch</code> calls in each page, or adopting <code>axios</code> .	No retries, no automatic re-authentication (an expired token routes the user to sign-in; the app never mints its own token), no request caching or offline queueing, and no CORS configuration — cross-origin access is the backend's <code>FRONTEND_ORIGIN</code> allowlist decision.
Answering an API refusal and where the persona sees it	<code>src/context/FeedbackContext.jsx</code> (a queue of <code>show(message, kind) / dismiss(id)</code>) and <code>src/components/FeedbackBanner.jsx</code> , rendered inside both persona layouts and the login screen.	The context sits above the router, so a refusal raised on one screen survives a client-side redirect and still lands where the user is; the banner inside each layout means every protected screen gets the same display. <code>window.alert</code> blocks the page and cannot be restyled; local state would vanish on navigation and duplicate the banner markup in every page. Alternative: Per-component local alert state, or <code>window.alert</code> .	Not a global toast system: no per-message expiry timer, no stacking limit beyond the list, no deduplication of identical back-to-back refusals, no translations or event tracking.
Gating what a persona can reach and is offered	<code>src/auth/RequireRole.jsx</code> wraps each persona's nested routes: <code>unauthenticated → /login</code> (remembering the intended path); wrong persona → that persona's own home.	A diner who types <code>/vendor/menu</code> gets an instant redirect instead of a screen full of refusal banners, and an unauthenticated visitor never sees a persona layout. The gate is explicitly not a security boundary: the backend re-verifies the token and role on every request, and Task 7's live check confirmed a diner's token is refused 403 forbidden on <code>GET /api/vendor/menu</code> even while the UI is running. Alternative: No gate at all — let any signed-in user reach any route and let every API call 403.	No protection against crafted URLs, direct API calls, or a token whose role the UI misread — the 401/403 boundary, record ownership and the stall relationship are enforced only in <code>app/auth.py</code> and the models. The gate also cannot see token expiry itself; it reacts when an API call's 401 clears the session.

Decision	Where it lives — module/component	Why there, against the alternative	What it does not give you
Request-in-flight duplicate submission control	src/components/AsyncButton.jsx — while the wrapped request is in flight the button is disabled and shows progress text; a second click cannot start a second request. useAsyncAction exposes the same guard for screens that drive their own control.	One shared pending behaviour (disabled + progress text) instead of per-page duplication, and the refusal-then-retry path (e.g. checkout's failed payment) keeps exactly one checkout key with a synchronous re-entry guard, so a double-click cannot queue two payments from this tab. Alternative: Ad-hoc submitting state in each page, or no guard (trusting the user not to double-click).	This is UI-side duplicate control for this tab's own clicks: a second tab, a second browser, a browser-level retry, or a network-level resend of the same request is invisible to the button. Those cases are handled at the database layer instead — the (diner, checkout_key) unique index makes a repeated checkout store at most one order, a matching replay returns the stored order (200) and a conflicting reuse is refused (409), as proven by the functional suite's two-thread race. The button prevents the user's intent from firing twice; the index prevents a duplicate row.

Each decision names the module or component, the real alternative it was weighed against, why it was chosen, and what it does not provide.

Token state and logout

src/auth/AuthContext.jsx stores {user, token} in sessionStorage (per-tab, so one browser can hold a diner in one tab and a vendor in another). login exchanges the seeded account's credentials through POST /api/user/gettoken; logout clears the tab's entry; any authentication_required 401 on a bound request clears it, which makes RequireRole route the user back to sign-in. Tokens are never stored in localStorage and are never read after logout.

Verification (7 October 2026)

- npm run build (Node 22.17.0, react-scripts 5.0.1): Compiled successfully, no ESLint warnings; production bundle rebuilt after the final change.
- Manual two-persona verification against the live dev API (5001) and the CRA dev server (5173), script .local/verification/task7_ui_check.py (Playwright, not an assessed suite) — all 10 checks passed, run twice (second run after a UI field-name fix): unauthenticated diner visit redirects to /login; wrong password shows the backend's refusal in the banner; diner login lands on the open-stall list with Charcoal Grill marked Open; cart and orders load under the token; a diner visiting /vendor/menu is redirected client-side and the backend refuses the same API call with the diner token (403 forbidden); a forged token's 401 clears the session and presents sign-in; the unknown path shows the not-found view; vendor login lands on the menu and the paid queue loads with the seeded orders.
- The first run's screenshot review caught a real UI bug: the stall trading-state badge read stall.open while the API field is is_open, so an open stall rendered "Closed". Fixed in StallsPage/VendorMenuPage and re-verified (badge now reads "Open"; the check now asserts the green badge).
- No frontend unit/component suite exists, as the TMA does not require one; the assessed browser suite is Task 11's Playwright test.

Appendix F — Persona flow screenshots

SkipQ sign-in

Email

diner.one@skipq.test

Password

Sign in

Demo accounts

(seeded; shared local password SkipQDemo2026!):

- Diner — diner.one@skipq.test (has current and past orders)
- Diner — diner.two@skipq.test
- Diner — diner.empty@skipq.test (no orders)
- Vendor — vendor.one@skipq.test (Charcoal Grill)
- Vendor — vendor.two@skipq.test (Noodle Bar)

Figure F1. task7 01 login
Source: docs/evidence/screenshots/task7-01-login.png

SkipQ Diner — diner.one@skipq.test

Sign out

Stalls

Refresh

Charcoal Grill Open

View menu

Figure F2. Diner login lands on the open-stall list (Charcoal Grill, Open)
Source: docs/evidence/screenshots/task8-01-stalls.png

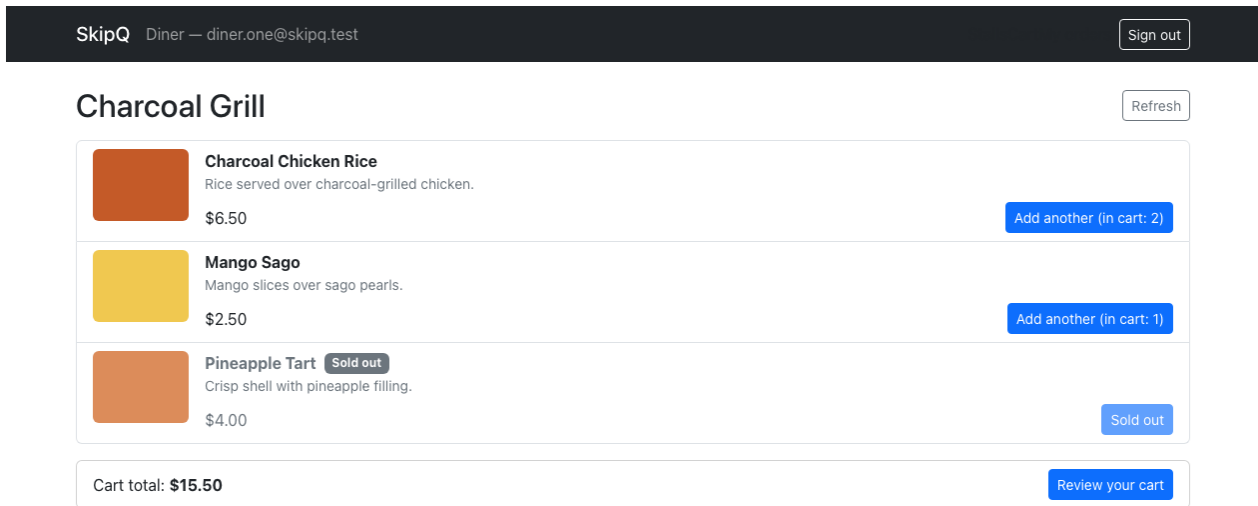


Figure F3. Seeded sold-out Pineapple Tart greyed with the disabled "Sold out" control
Source: docs/evidence/screenshots/task8-02-menu-sold-out.png

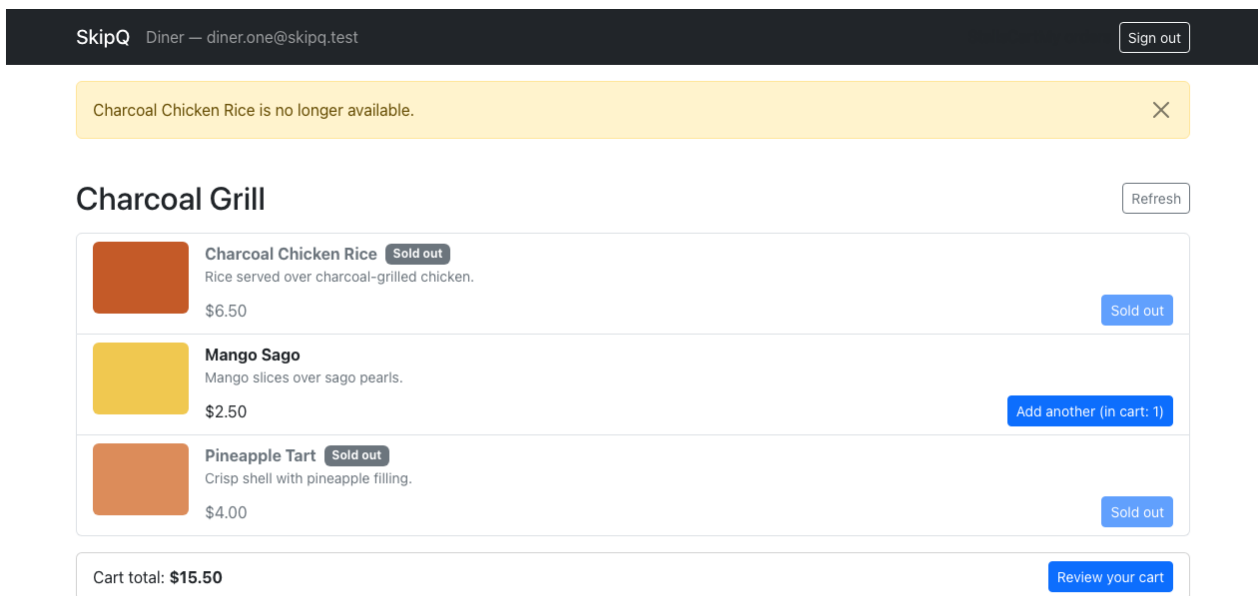


Figure F4. Toast for the newly observed sold-out change, menu still listing the item
Source: docs/evidence/screenshots/task8-03-sold-out-toast.png

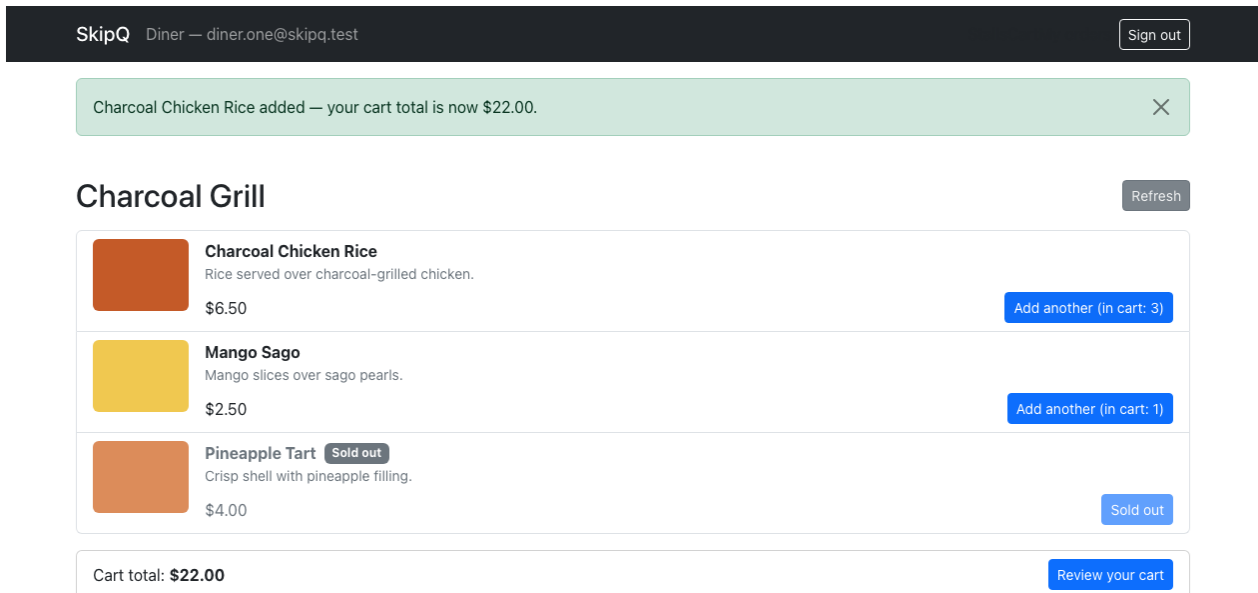


Figure F5. "Add another (in cart: 2)" increment confirmed; cart total card \$22.00
Source: docs/evidence/screenshots/task8-04-menu-after-add.png

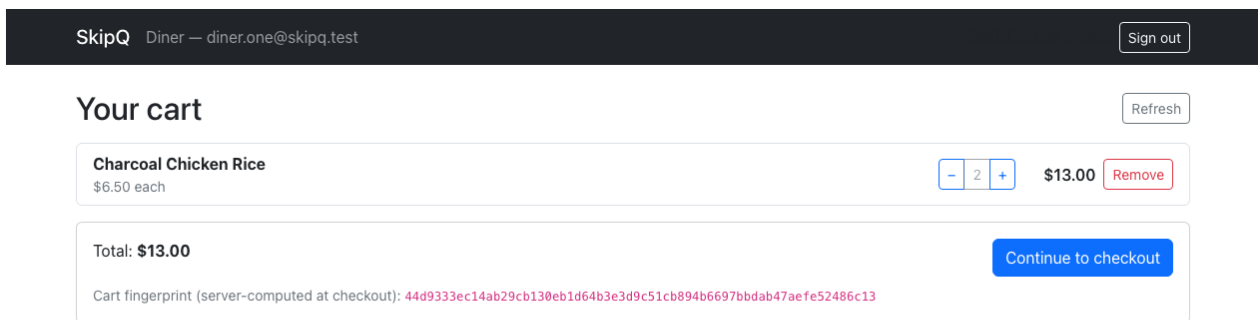


Figure F6. Cart after stepper/removal edits: 2× M1, total \$13.00, fingerprint shown
Source: docs/evidence/screenshots/task8-05-cart-edited.png

SkipQDiner — diner.one@skipq.test

Sign out

Checkout

Refresh

2 × Charcoal Chicken Rice

\$13.00

Total: \$13.00

[Edit cart](#)

Cart fingerprint: 44d9333ec14ab29cb130eb1d64b3e3d9c51cb894b6697bbdab47aefe52486c13

Payment method (simulated)

☒ PayNow

☐ Card

☐ Cashless

☐ Simulate this payment failing (demo control)

Pay \$13.00

Figure F7. Checkout summary, 64-char server fingerprint, PayNow selected
Source: docs/evidence/screenshots/task8-06-checkout.png

SkipQDiner — diner.one@skipq.test

Sign out

Checkout refused

Your cart total changed since it was loaded. Review your cart and try checkout again.

Your checkout key is retained for this attempt; reviewing or refreshing the cart will re-send the server's current fingerprint.

Review your cart

Refresh and retry

Figure F8. Stale-cart refusal ("Your cart total changed since it was loaded...") with Review / Refresh-and-retry and the retained-key notice
Source: docs/evidence/screenshots/task8-07-checkout-stale-refused.png

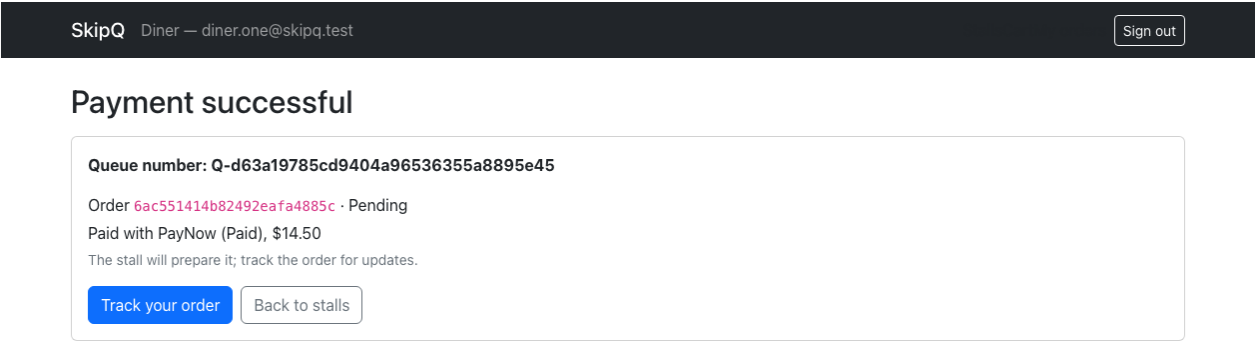


Figure F9. Payment successful: queue number, order id, Pending, PayNow Paid, \$14.50
Source: docs/evidence/screenshots/task8-08-checkout-success.png

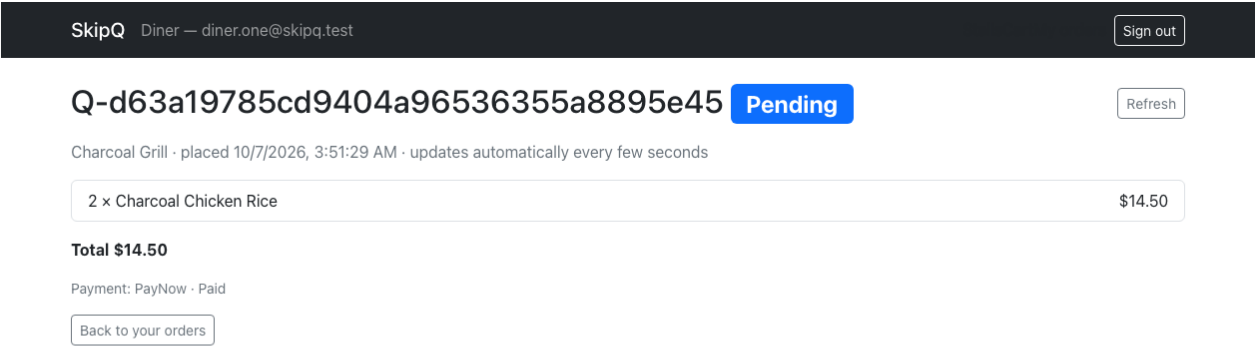


Figure F10. Tracking page: queue number, Pending badge, snapshot lines, payment state
Source: docs/evidence/screenshots/task8-09-tracking-pending.png

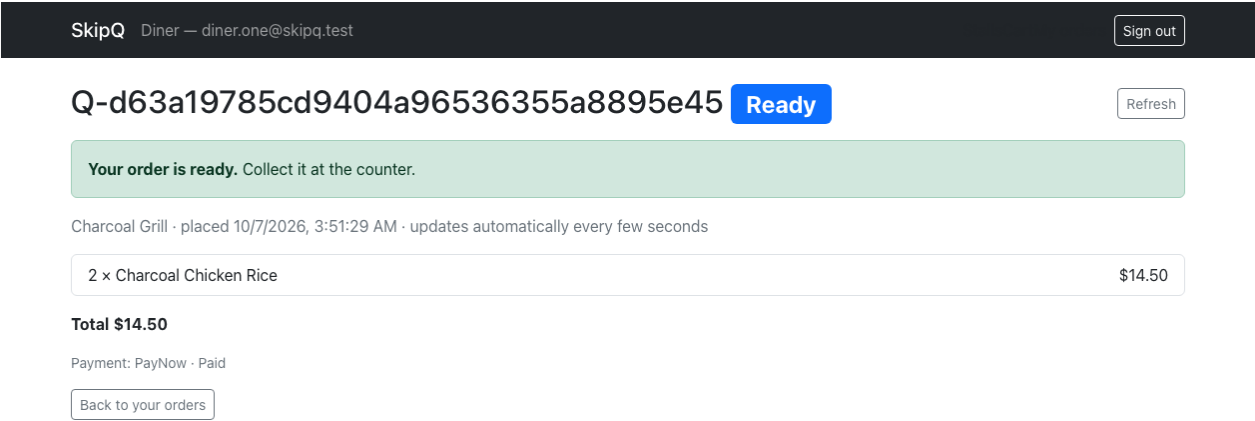


Figure F11. Ready badge with the green "Your order is ready. Collect it at the counter." banner
Source: docs/evidence/screenshots/task8-10-tracking-ready.png

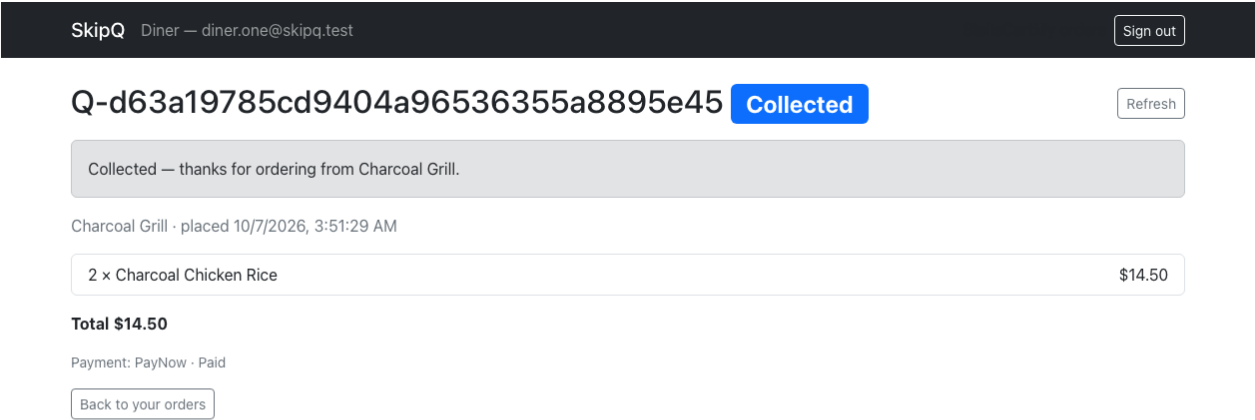


Figure F12. Terminal Collected notice (polling now stopped)
Source: docs/evidence/screenshots/task8-11-tracking-collected.png

SkipQ

Diner — diner.one@skipq.test

Sign out

Checkout

Refresh

1 x Mango Sago

\$2.50

Total: **\$2.50**

[Edit cart](#)

Cart fingerprint: e1985ac08c0c07045d3849c36f96a8e5bb3dbdb61fd256b5fcc3d3689eab3673

Payment was not completed. Your cart is unchanged; you can try again. The same checkout key is retained — retrying will not create a duplicate order.

Payment method (simulated)

☒ PayNow

☐ Card

☐ Cashless

☒ Simulate this payment failing (demo control)

Pay \$2.50

Figure F13. Simulated payment failure with the retained-checkout-key retry notice
Source: docs/evidence/screenshots/task8-12-payment-failed.png

SkipQ

Diner — diner.one@skipq.test

Sign out

Payment successful

Queue number: Q-f9c100013f5c4277abf15a23b646275c

Order 6ac5514b4b82492eafa4885d · Pending

Paid with PayNow (Paid), \$2.50

The stall will prepare it; track the order for updates.

[Track your order](#)

[Back to stalls](#)

Figure F14. Retry under the same key: payment successful, one order stored
Source: docs/evidence/screenshots/task8-13-payment-retry-success.png

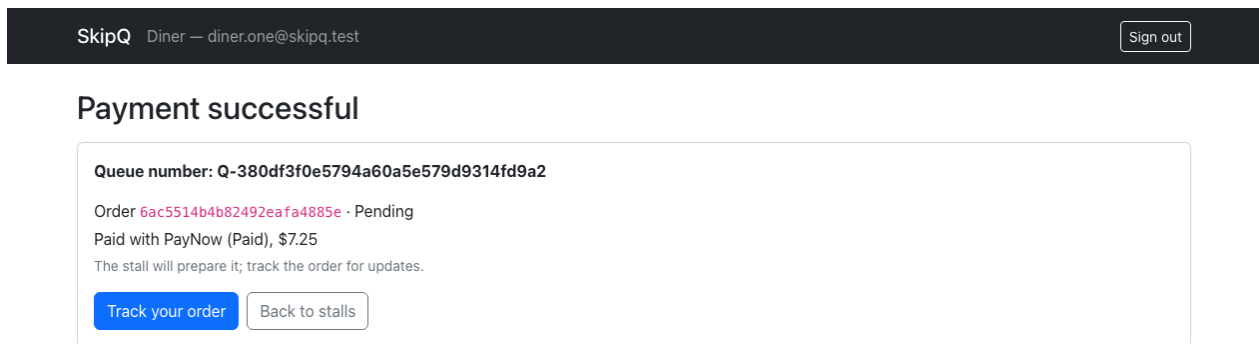


Figure F15. Double-click result screen; exactly one order exists for the attempt
Source: docs/evidence/screenshots/task8-14-double-click-single-order.png

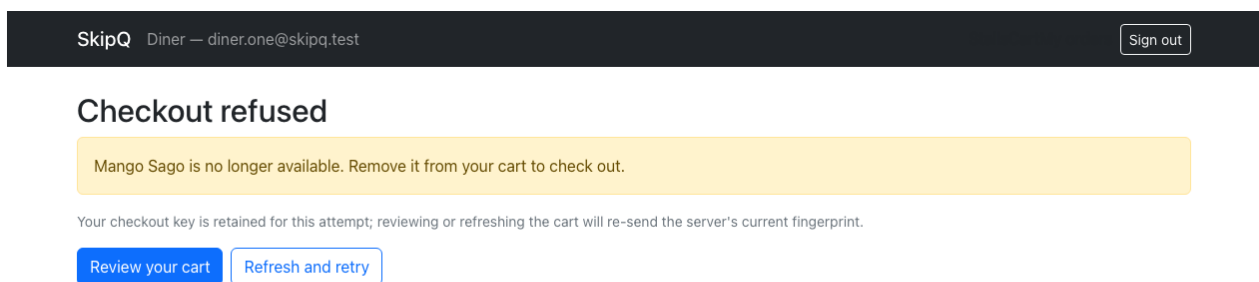


Figure F16. Checkout refused because a line sold out before payment
Source: docs/evidence/screenshots/task8-15-stale-sold-out-refused.png

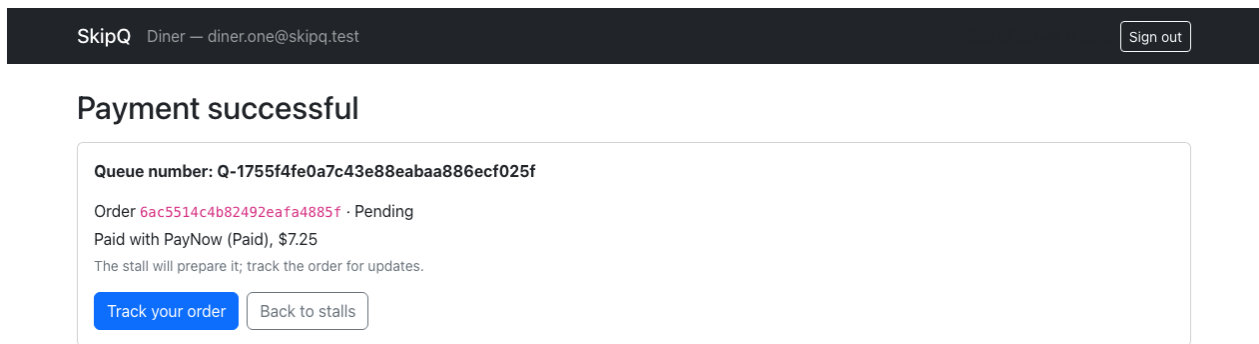


Figure F17. Remaining line paid out after the sold-out line was removed
Source: docs/evidence/screenshots/task8-16-after-removal-paid.png

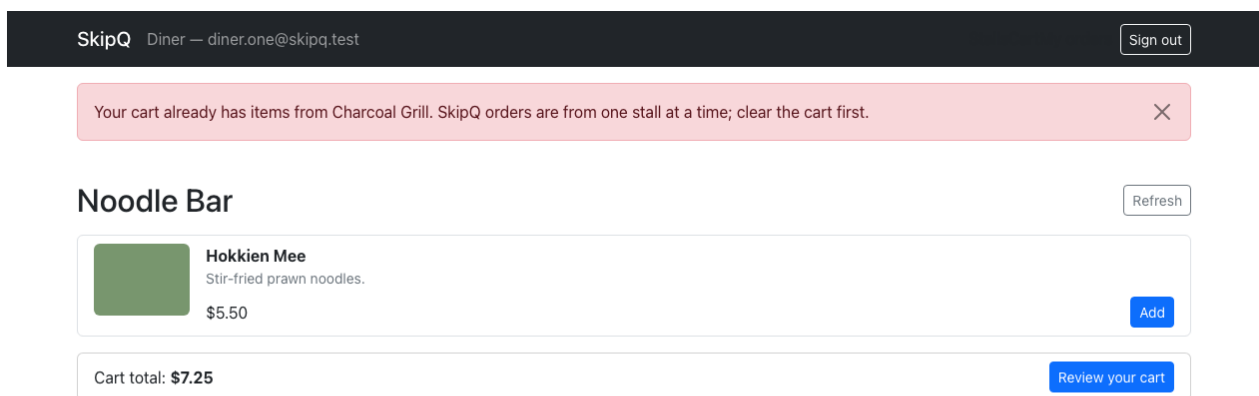


Figure F18. Cross-stall add refusal naming the stall already in the cart
Source: docs/evidence/screenshots/task8-17-cross-stall-refused.png

SkipQ sign-in

Sign in again to continue. X

Email

diner.one@skipq.test

Password

Sign in

Demo accounts (seeded; shared local password SkipQDemo2026!):

- Diner — diner.one@skipq.test (has current and past orders)
- Diner — diner.two@skipq.test
- Diner — diner.empty@skipq.test (no orders)
- Vendor — vendor.one@skipq.test (Charcoal Grill)
- Vendor — vendor.two@skipq.test (Noodle Bar)

Figure F19. Forged-token 401: session cleared, sign-in presented
Source: docs/evidence/screenshots/task8-18-forged-token-sign-in.png

SkipQ Vendor — vendor.one@skipq.test Sign out

Charcoal Grill Open

Refresh Add menu item

Trading state — open; diners can order here

Charcoal Chicken Rice

Rice served over charcoal-grilled chicken.

\$6.50 · updated 10/7/2026, 3:51:29 AM

Mark sold out Edit Remove

Mango Sago

Mango slices over sago pearls.

\$2.50 · updated 10/7/2026, 4:25:53 AM

Mark sold out Edit Remove

Pineapple Tart

Crisp shell with pineapple filling.

\$4.00 · updated 10/7/2026, 1:14:11 AM

Restock Edit Remove

Figure F20. Vendor login lands on the own menu: stall name, Open badge, 3 active items, seeded Pineapple Tart badged Sold out
Source: docs/evidence/screenshots/task9-01-vendor-menu-open.png

SkipQ Vendor — vendor.one@skipq.test
Sign out

The stall is now closed; diners cannot add to a cart here, and the paid queue stays accessible.

Charcoal Grill Closed

Trading state — closed; diners cannot order here

This stall is closed, so new orders are refused at checkout. Every paid order in [your order queue](#) stays accessible.

	Charcoal Chicken Rice Rice served over charcoal-grilled chicken.	\$6.50 · updated 10/7/2026, 3:51:29 AM
Mark sold out Edit Remove		
	Mango Sago Mango slices over sago pearls.	\$2.50 · updated 10/7/2026, 4:25:53 AM
Mark sold out Edit Remove		
	Pineapple Tart Sold out Crisp shell with pineapple filling.	\$4.00 · updated 10/7/2026, 1:14:11 AM
Restock Edit Remove		

Figure F21. After the switch: Closed badge and the closure banner (new orders refused, paid queue stays accessible)
Source: docs/evidence/screenshots/task9-02-closed-banner.png

SkipQ Diner — diner.one@skipq.test
Sign out

Charcoal Grill

Refresh

Charcoal Grill is currently closed. Try again once the stall opens.

	Charcoal Chicken Rice Rice served over charcoal-grilled chicken.	\$6.50	Add another (in cart: 2)
	Mango Sago Mango slices over sago pearls.	\$2.50	Add another (in cart: 1)
	Pineapple Tart Sold out Crisp shell with pineapple filling.	\$4.00	Sold out

Cart total: **\$15.50**
Review your cart

Figure F22. The diner's direct-link view of the closed stall: closure notice, every Add control disabled
Source: docs/evidence/screenshots/task9-03-diner-closed-stall.png

SkipQVendor — vendor.one@skipq.testSign out

Price must be a number above 0, below 9999, with at most two decimal places.

Add menu item

Rules: name 1–80 characters; price above 0 and below 9999 with at most two decimal places; description up to 500 characters; image an http(s) URL for a JPG, JPEG, or PNG.

Name

Task Nine Tiao

Price

abc

Description

What the dish is.

Image URL

https://example.com/images/task-nine-tiao.jpg

☒ Available now (diners can add it to a cart)

Figure F23. The create form showing the backend's own 400 message for a malformed price
Source: docs/evidence/screenshots/task9-04-form-rejection.png

SkipQVendor — vendor.one@skipq.testSign out

Task Nine Tiao was added to your menu.

Charcoal Grill

Open

RefreshAdd menu item

☒ Trading state — open; diners can order here

Charcoal Chicken Rice

Rice served over charcoal-grilled chicken.

Mark sold out

Edit

Remove

\$6.50 · updated 10/7/2026, 3:51:29 AM

Mango Sago

Mango slices over sago pearls.

Mark sold out

Edit

Remove

\$2.50 · updated 10/7/2026, 4:25:53 AM

Pineapple Tart

Sold out

Crisp shell with pineapple filling.

Restock

Edit

Remove

\$4.00 · updated 10/7/2026, 1:14:11 AM

Task Nine Tiao

A noodle dish created by the Task 9 verification run.

Mark sold out

Edit

Remove

\$9.99 · updated 10/7/2026, 4:26:20 AM

Figure F24. The run-created item listed with its \$9.99 price and the success banner
Source: docs/evidence/screenshots/task9-05-item-created.png

Page 35

SkipQVendor — vendor.one@skipq.testSign out

Task Nine Tiao was added to your menu.

Task Nine Tiao Soup was updated.

Charcoal GrillOpen

RefreshAdd menu item

Trading state — open; diners can order here

Charcoal Chicken Rice

Rice served over charcoal-grilled chicken.

Mark sold out

Edit

Remove

Mango Sago

Mango slices over sago pearls.

Mark sold out

Edit

Remove

Pineapple Tart

Sold out

Crisp shell with pineapple filling.

Restock

Edit

Remove

Task Nine Tiao Soup

A noodle dish created by the Task 9 verification run.

Mark sold out

Edit

Remove

\$6.50 · updated 10/7/2026, 3:51:29 AM

\$2.50 · updated 10/7/2026, 4:25:53 AM

\$4.00 · updated 10/7/2026, 1:14:11 AM

\$10.50 · updated 10/7/2026, 4:26:20 AM

Figure F25. The renamed/repriced item (\$10.50) after the edit form saved
Source: docs/evidence/screenshots/task9-06-item-edited.png

SkipQVendor — vendor.one@skipq.testSign out

Task Nine Tiao Soup was updated.

Mango Sago is now marked sold out.

Charcoal GrillOpen

RefreshAdd menu item

Trading state — open; diners can order here

Charcoal Chicken Rice

Rice served over charcoal-grilled chicken.

Mark sold out

Edit

Remove

Mango Sago

Sold out

Mango slices over sago pearls.

Restock

Edit

Remove

Pineapple Tart

Sold out

Crisp shell with pineapple filling.

Restock

Edit

Remove

Task Nine Tiao Soup

Task Nine Tiao Soup

A noodle dish created by the Task 9 verification run.

Mark sold out

Edit

Remove

\$6.50 · updated 10/7/2026, 3:51:29 AM

\$2.50 · updated 10/7/2026, 4:26:21 AM

\$4.00 · updated 10/7/2026, 1:14:11 AM

\$10.50 · updated 10/7/2026, 4:26:20 AM

Figure F26. A seeded item marked sold out from its row control (badge + success banner)
Source: docs/evidence/screenshots/task9-07-m2-sold-out.png

Page 36

SkipQVendor — vendor.one@skipq.testSign out

Orders

Refresh

Q-seed-0003 Ready 1 item · \$5.00 · 10/7/2026, 4:06:17 AM	Manage
Q-seed-0002 Preparing 2 items · \$9.00 · 10/7/2026, 4:01:17 AM	Manage
Q-seed-0001 Pending 1 item · \$13.00 · 10/7/2026, 3:56:17 AM	Manage
Q-seed-0004 Ready 2 items · \$9.00 · 10/7/2026, 3:46:17 AM	Manage
Q-seed-0007 NoShow 2 items · \$9.00 · 10/6/2026, 11:26:17 PM	Manage
Q-seed-0006 Cancelled 1 item · \$2.50 · 10/5/2026, 4:26:17 AM	Manage
Q-seed-0005 Collected 1 item · \$6.00 · 10/4/2026, 4:26:17 AM	Manage
Q-seed-0008 Collected 1 item · \$5.00 · 10/1/2026, 4:26:17 AM	Manage

Figure F27. The paid queue: 8 own-stall orders newest-first, other-stall order absent, status badges
Source: docs/evidence/screenshots/task9-08-orders-queue.png

SkipQVendor — vendor.one@skipq.testSign out

Q-seed-0001

Pending

Refresh

Charcoal Grill · placed 10/7/2026, 3:56:17 AM · updates automatically every few seconds

2 × Charcoal Chicken Rice

\$13.00

Total \$13.00

Payment: PayNow · Paid · \$13.00

Accept order

Reject order

Back to your order queue

Figure F28. A Pending order: exactly Accept order / Reject order
Source: docs/evidence/screenshots/task9-09-pending-controls.png

SkipQ

Vendor — vendor.one@skipq.test

Sign out

Q-seed-0002

Preparing

Refresh

Charcoal Grill · placed 10/7/2026, 4:01:17 AM · updates automatically every few seconds

1 × Charcoal Chicken Rice	\$6.50
1 × Mango Sago	\$2.50

Total \$9.00

Payment: Card · Paid · \$9.00

Mark ready

Back to your order queue

Figure F29. A Preparing order: exactly Mark ready (rejection no longer offered)
Source: docs/evidence/screenshots/task9-10-preparing-controls.png

SkipQ

Vendor — vendor.one@skipq.test

Sign out

Q-seed-0003

Ready

Refresh

Charcoal Grill · placed 10/7/2026, 4:06:17 AM · ready 10/7/2026, 4:21:17 AM · updates automatically every few seconds

2 × Mango Sago	\$5.00
----------------	--------

Total \$5.00

Payment: Cashless · Paid · \$5.00

Mark collected

Back to your order queue

Figure F30. Ready +5 min: Mark collected only — the NoShow control is still hidden
Source: docs/evidence/screenshots/task9-11-ready-no-noshow.png

SkipQ

Vendor — vendor.one@skipq.test

Sign out

Q-seed-0004

Ready

Refresh

Charcoal Grill · placed 10/7/2026, 3:46:17 AM · ready 10/7/2026, 3:51:17 AM · updates automatically every few seconds

1 × Charcoal Chicken Rice	\$6.50
1 × Mango Sago	\$2.50

Total \$9.00

Payment: PayNow · Paid · \$9.00

Mark collected

Mark no-show

Back to your order queue

Figure F31. Ready +35 min: both Mark collected and Mark no-show are offered
Source: docs/evidence/screenshots/task9-12-ready-with-noshow.png

SkipQ

Vendor — vendor.one@skipq.test

Sign out

Q-seed-0006

Cancelled

Refresh

This order was rejected. The PayNow payment of \$2.50 was refunded on 10/5/2026, 4:36:17 AM

Charcoal Grill · placed 10/5/2026, 4:26:17 AM

1 × Mango Sago	\$2.50
----------------	--------

Total \$2.50

Payment: PayNow · Refunded · \$2.50 · refunded 10/5/2026, 4:36:17 AM

This order is final; it can no longer be changed.

Back to your order queue

Figure F32. Cancelled order: read-only, "This order is final", refund shown with time
Source: docs/evidence/screenshots/task9-13-terminal-cancelled.png

SkipQ

Vendor — vendor.one@skipq.test

Sign out

Q-seed-0007

NoShow

Refresh

The diner did not collect. The Cashless payment of \$9.00 stays paid.

Charcoal Grill · placed 10/6/2026, 11:26:17 PM · ready 10/7/2026, 12:26:17 AM

1 × Charcoal Chicken Rice	\$6.50
1 × Mango Sago	\$2.50

Total \$9.00

Payment: Cashless · Paid · \$9.00

This order is final; it can no longer be changed.

Back to your order queue

Figure F33. NoShow order: read-only, payment-stays-paid notice
Source: docs/evidence/screenshots/task9-14-terminal-noshow.png

SkipQ

Vendor — vendor.one@skipq.test

Sign out

Q-seed-0005

Collected

Refresh

Collected — this order is complete and can no longer be changed.

Charcoal Grill · placed 10/4/2026, 4:26:17 AM

1 × Grilled Chicken Rice	\$6.00
--------------------------	--------

Total \$6.00

Payment: Card · Paid · \$6.00

This order is final; it can no longer be changed.

Back to your order queue

Figure F34. Collected order: read-only, stale purchase snapshot (Grilled Chicken Rice \$6.00)
Source: docs/evidence/screenshots/task9-15-terminal-collected-snapshot.png

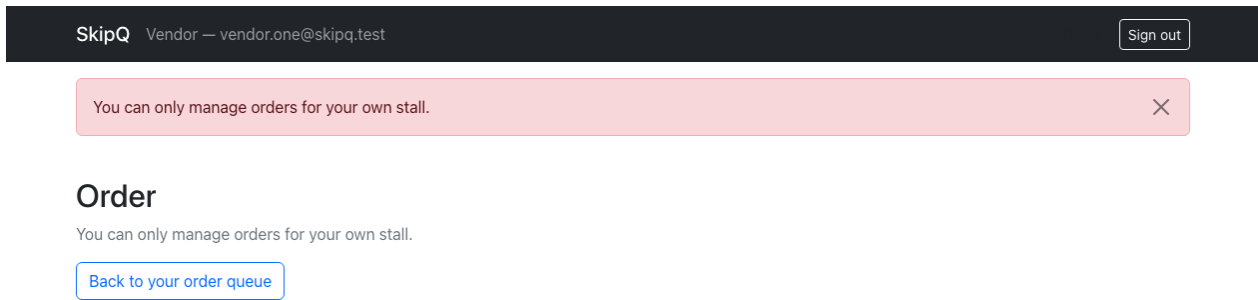


Figure F35. Another stall's order: the 403 screen, no order data rendered
Source: docs/evidence/screenshots/task9-16-forbidden-order.png

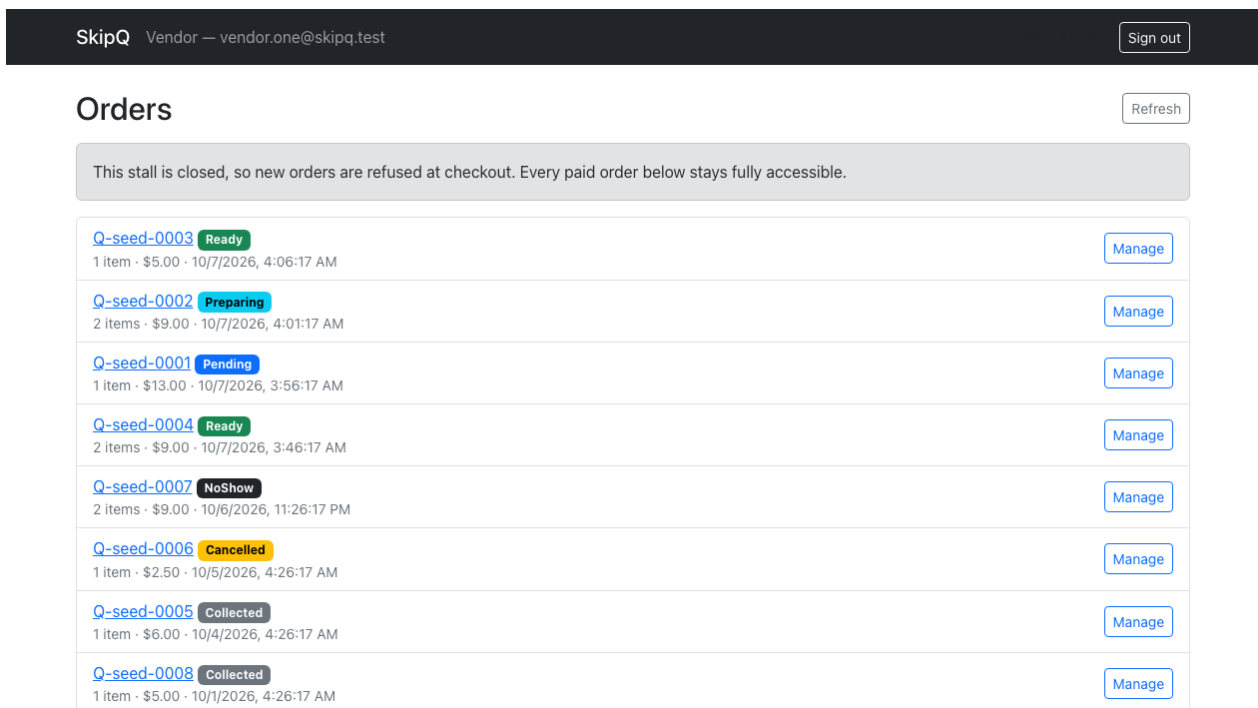


Figure F36. While the stall is closed: closure banner plus the full paid queue with every Manage control enabled
Source: docs/evidence/screenshots/task9-17-closed-queue-accessible.png

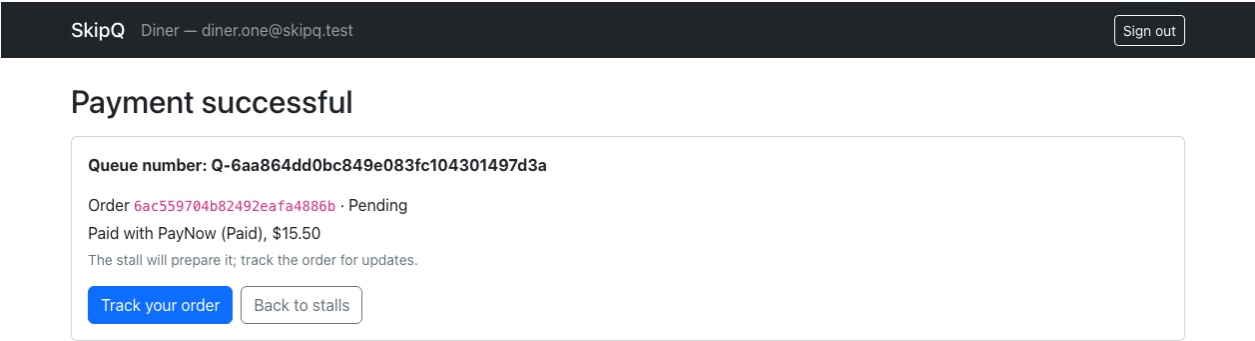


Figure F37. The diner context: the seeded cart (\$15.50) paid, fresh queue number shown
Source: docs/evidence/screenshots/task9-18-diner-checkout-success.png

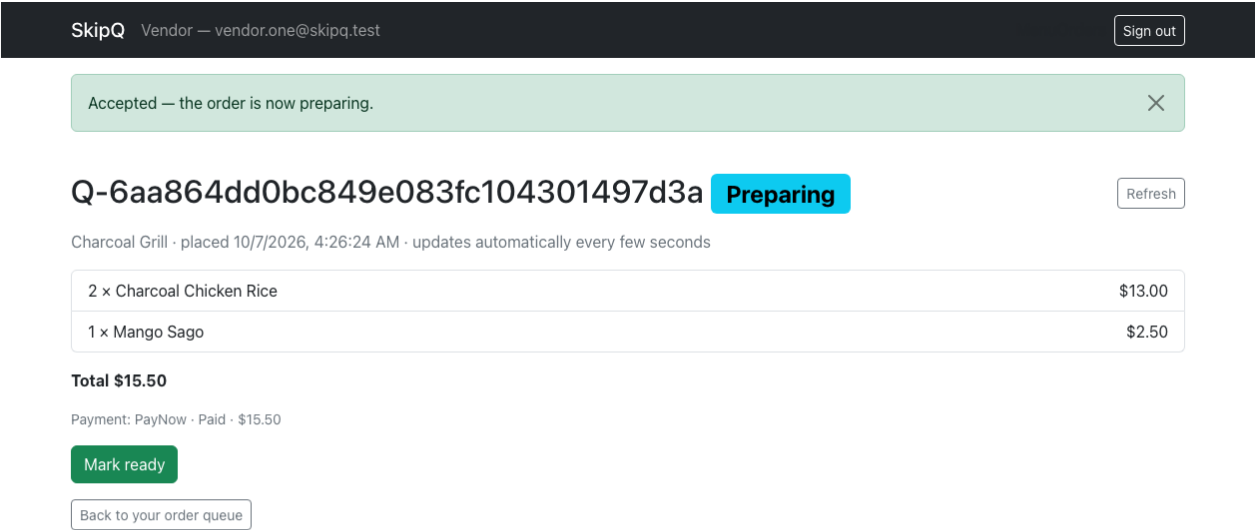


Figure F38. The vendor detail of the same order after Accept: Preparing badge, success notice
Source: docs/evidence/screenshots/task9-19-vendor-accepted.png

SkipQ

Diner — diner.one@skipq.test

Sign out

Q-6aa864dd0bc849e083fc104301497d3a

Preparing

Refresh

Charcoal Grill · placed 10/7/2026, 4:26:24 AM · updates automatically every few seconds

2 × Charcoal Chicken Rice	\$13.00
1 × Mango Sago	\$2.50

Total \$15.50

Payment: PayNow · Paid

Back to your orders

Figure F39. The diner's tracking page observing Preparing (polling, no manual refresh)
Source: docs/evidence/screenshots/task9-20-diner-preparing.png

SkipQ

Diner — diner.one@skipq.test

Sign out

Q-6aa864dd0bc849e083fc104301497d3a

Ready

Refresh

Your order is ready. Collect it at the counter.

Charcoal Grill · placed 10/7/2026, 4:26:24 AM · updates automatically every few seconds

2 × Charcoal Chicken Rice	\$13.00
1 × Mango Sago	\$2.50

Total \$15.50

Payment: PayNow · Paid

Back to your orders

Figure F40. The Ready-for-collection banner on the diner's tracking page after Mark ready
Source: docs/evidence/screenshots/task9-21-diner-ready-banner.png

SkipQVendor — vendor.one@skipq.test

Sign out

Accepted — the order is now preparing.

Marked ready; the diner can collect it at the counter.

Marked collected — the order is complete.

Q-6aa864dd0bc849e083fc104301497d3a

Collected

Refresh

Collected — this order is complete and can no longer be changed.

Charcoal Grill · placed 10/7/2026, 4:26:24 AM · ready 10/7/2026, 4:26:24 AM

2 × Charcoal Chicken Rice	\$13.00
1 × Mango Sago	\$2.50

Total \$15.50

Payment: PayNow · Paid · \$15.50

This order is final; it can no longer be changed.

Back to your order queue

Figure F41. The vendor detail after Mark collected: terminal, read-only notice
Source: docs/evidence/screenshots/task9-22-vendor-collected.png

SkipQDiner — diner.one@skipq.test

Sign out

Q-6aa864dd0bc849e083fc104301497d3a

Collected

Refresh

Collected — thanks for ordering from Charcoal Grill.

Charcoal Grill · placed 10/7/2026, 4:26:24 AM

2 × Charcoal Chicken Rice	\$13.00
1 × Mango Sago	\$2.50

Total \$15.50

Payment: PayNow · Paid

Back to your orders

Figure F42. The diner's terminal Collected notice for the same order
Source: docs/evidence/screenshots/task9-23-diner-collected.png

SkipQ Vendor — vendor.one@skipq.test Sign out

Rejected; the payment has been refunded. ×

Q-seed-0001 **Cancelled** Refresh

This order was rejected. The PayNow payment of \$13.00 was refunded on 10/7/2026, 4:26:31 AM

Charcoal Grill · placed 10/7/2026, 3:56:17 AM

2 × Charcoal Chicken Rice	\$13.00
---------------------------	---------

Total \$13.00

Payment: PayNow · Refunded · \$13.00 · refunded 10/7/2026, 4:26:31 AM

This order is final; it can no longer be changed.

Back to your order queue

Figure F43. The vendor rejecting the seeded Pending order: Cancelled badge, refund shown
Source: docs/evidence/screenshots/task9-24-cancel-refund.png

SkipQ Diner — diner.one@skipq.test Sign out

Q-seed-0001 **Cancelled** Refresh

This order was cancelled and its PayNow payment of \$13.00 was refunded.

Charcoal Grill · placed 10/7/2026, 3:56:17 AM

2 × Charcoal Chicken Rice	\$13.00
---------------------------	---------

Total \$13.00

Payment: PayNow · Refunded · refunded 10/7/2026, 4:26:31 AM

Back to your orders

Figure F44. The diner's cancellation notice with the refunded \$13.00 amount
Source: docs/evidence/screenshots/task9-25-diner-cancelled.png

SkipQ Vendor — vendor.one@skipq.test Sign out

Marked no-show; the payment stays paid. ×

Q-seed-0004 **NoShow** Refresh

The diner did not collect. The PayNow payment of \$9.00 stays paid.

Charcoal Grill · placed 10/7/2026, 3:46:17 AM · ready 10/7/2026, 3:51:17 AM

1 × Charcoal Chicken Rice	\$6.50
1 × Mango Sago	\$2.50

Total \$9.00

Payment: PayNow · Paid · \$9.00

This order is final; it can no longer be changed.

Back to your order queue

Figure F45. Mark no-show accepted past the 30-minute boundary: NoShow badge, stays-paid notice
Source: docs/evidence/screenshots/task9-26-vendor-noshow.png

SkipQ Diner — diner.one@skipq.test Sign out

Q-seed-0004 **NoShow** Refresh

This order was not collected and was marked No Show. The payment stays paid.

Charcoal Grill · placed 10/7/2026, 3:46:17 AM

1 × Charcoal Chicken Rice	\$6.50
1 × Mango Sago	\$2.50

Total \$9.00

Payment: PayNow · Paid

Back to your orders

Figure F46. The diner's no-show notice ("The payment stays paid.")
Source: docs/evidence/screenshots/task9-27-diner-noshow.png

SkipQ Diner — diner.one@skipq.test

Sign out

My orders

AllCurrentPastRefresh

Q-seed-0009 Pending	Noodle Bar · 10/7/2026, 4:27:31 AM · \$11.00	Details
Q-seed-0003 Ready	Charcoal Grill · 10/7/2026, 4:22:31 AM · \$5.00	Details
Q-seed-0002 Preparing	Charcoal Grill · 10/7/2026, 4:17:31 AM · \$9.00	Details
Q-seed-0001 Pending	Charcoal Grill · 10/7/2026, 4:12:31 AM · \$13.00	Details
Q-seed-0004 Ready	Charcoal Grill · 10/7/2026, 4:02:31 AM · \$9.00	Details
Q-seed-0007 NoShow	Charcoal Grill · 10/6/2026, 11:42:31 PM · \$9.00	Details
Q-seed-0006 Cancelled	Charcoal Grill · 10/5/2026, 4:42:31 AM · \$2.50	Details
Q-seed-0005 Collected	Charcoal Grill · 10/4/2026, 4:42:31 AM · \$6.00	Details

Figure F47. All (default): the diner's 8 own orders across all six statuses, newest-first, another diner's order absent
Source: docs/evidence/screenshots/task10-01-orders-all.png

SkipQ Diner — diner.one@skipq.test

Sign out

My orders

AllCurrentPastRefresh

Q-seed-0009 Pending	Noodle Bar · 10/7/2026, 4:27:31 AM · \$11.00	Details
Q-seed-0003 Ready	Charcoal Grill · 10/7/2026, 4:22:31 AM · \$5.00	Details
Q-seed-0002 Preparing	Charcoal Grill · 10/7/2026, 4:17:31 AM · \$9.00	Details
Q-seed-0001 Pending	Charcoal Grill · 10/7/2026, 4:12:31 AM · \$13.00	Details
Q-seed-0004 Ready	Charcoal Grill · 10/7/2026, 4:02:31 AM · \$9.00	Details

Figure F48. Current filter: only the 5 Pending/Preparing/Ready orders
Source: docs/evidence/screenshots/task10-02-orders-current.png

SkipQ Diner — diner.one@skipq.test

Sign out

My orders

AllCurrentPastRefresh

Q-seed-0007NoShow

Charcoal Grill · 10/6/2026, 11:42:31 PM · \$9.00

Details

Q-seed-0006Cancelled

Charcoal Grill · 10/5/2026, 4:42:31 AM · \$2.50

Details

Q-seed-0005Collected

Charcoal Grill · 10/4/2026, 4:42:31 AM · \$6.00

Details

Figure F49. Past filter: exactly the 3 terminal orders, newest-first (this view never polls)
Source: docs/evidence/screenshots/task10-03-orders-past.png

SkipQ Diner — diner.one@skipq.test

Sign out

Q-seed-0005Collected

Refresh

Collected — thanks for ordering from Charcoal Grill.

Charcoal Grill · placed 10/4/2026, 4:42:31 AM

1 × Grilled Chicken Rice

\$6.00

Total \$6.00

Payment: Card · Paid

Back to your orders

Figure F50. Read-only past detail: Collected notice, stale snapshot (Grilled Chicken Rice \$6.00)
Source: docs/evidence/screenshots/task10-04-past-collected-snapshot.png

SkipQ Diner — diner.one@skipq.test Sign out

Q-seed-0006 Cancelled Refresh

This order was cancelled and its PayNow payment of \$2.50 was refunded.

Charcoal Grill · placed 10/5/2026, 4:42:31 AM

1 × Mango Sago	\$2.50
----------------	--------

Total \$2.50

Payment: PayNow · Refunded · refunded 10/5/2026, 4:52:31 AM

Back to your orders

Figure F51. Read-only past detail: Cancelled notice with the refunded \$2.50 and time
Source: docs/evidence/screenshots/task10-05-past-cancelled-refund.png

SkipQ Diner — diner.one@skipq.test Sign out

Q-seed-0001 Pending Refresh

Charcoal Grill · placed 10/7/2026, 4:12:31 AM · updates automatically every few seconds

2 × Charcoal Chicken Rice	\$13.00
---------------------------	---------

Total \$13.00

Payment: PayNow · Paid

Back to your orders

Figure F52. An in-progress order keeping its \$13.00 snapshot total after the menu item was live-repriced
Source: docs/evidence/screenshots/task10-06-current-snapshot-after-reprice.png



Figure F53. An order-free diner: "You have no orders yet." with the Browse open stalls link
Source: docs/evidence/screenshots/task10-07-empty-diner-all.png



Figure F54. The same diner on Past: "No past orders yet."
Source: docs/evidence/screenshots/task10-08-empty-diner-past.png

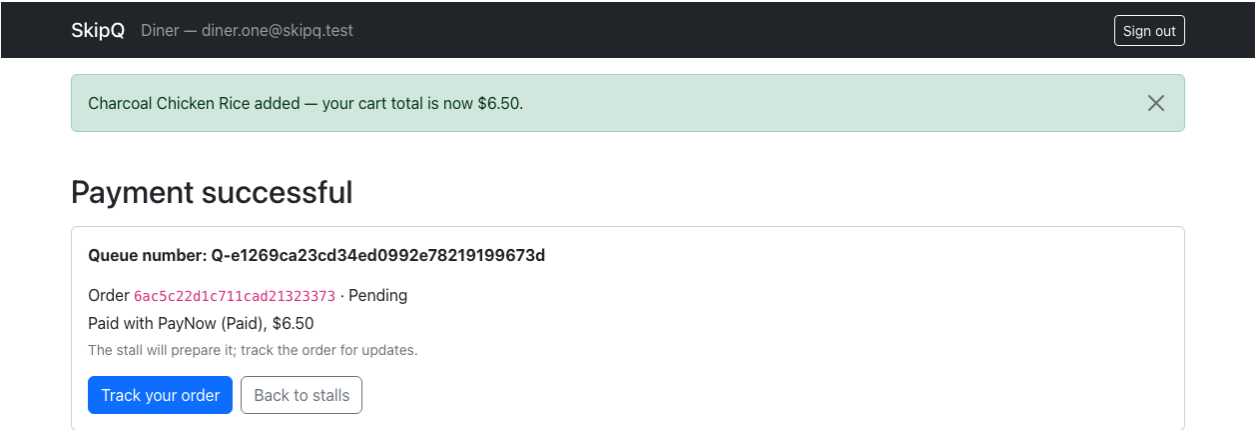


Figure F55. task11 01 diner payment success pending
Source: docs/evidence/screenshots/task11-01-diner-payment-success-pending.png

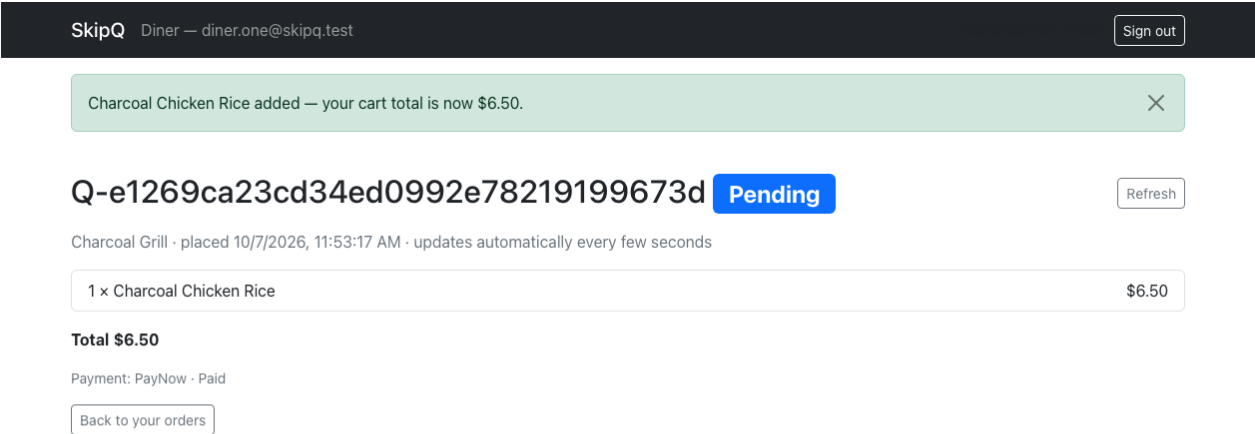


Figure F56. task11 02 diner pending
Source: docs/evidence/screenshots/task11-02-diner-pending.png

SkipQ Vendor — vendor.one@skipq.test		Sign out
Orders		Refresh
Q-e1269ca23cd34ed0992e78219199673d Pending		Manage
1 item · \$6.50 · 10/7/2026, 11:53:17 AM		
Q-seed-0003 Ready		Manage
1 item · \$5.00 · 10/7/2026, 11:13:33 AM		
Q-seed-0002 Preparing		Manage
2 items · \$9.00 · 10/7/2026, 11:08:33 AM		
Q-seed-0001 Pending		Manage
1 item · \$13.00 · 10/7/2026, 11:03:33 AM		
Q-seed-0004 Ready		Manage
2 items · \$9.00 · 10/7/2026, 10:53:33 AM		
Q-seed-0007 NoShow		Manage
2 items · \$9.00 · 10/7/2026, 6:33:33 AM		
Q-seed-0006 Cancelled		Manage
1 item · \$2.50 · 10/5/2026, 11:33:33 AM		
Q-seed-0005 Collected		Manage
1 item · \$6.00 · 10/4/2026, 11:33:33 AM		
Q-seed-0008 Collected		Manage
1 item · \$5.00 · 10/1/2026, 11:33:33 AM		

Figure F57. task11 03 vendor queue with new pending order

Source: docs/evidence/screenshots/task11-03-vendor-queue-with-new-pending-order.png

SkipQ Vendor — vendor.one@skipq.test		Sign out
Q-e1269ca23cd34ed0992e78219199673d Pending		Refresh
Charcoal Grill · placed 10/7/2026, 11:53:17 AM · updates automatically every few seconds		
1 x Charcoal Chicken Rice		\$6.50
Total \$6.50		
Payment: PayNow · Paid · \$6.50		
Accept order	Reject order	
Back to your order queue		

Figure F58. task11 04 vendor pending

Source: docs/evidence/screenshots/task11-04-vendor-pending.png

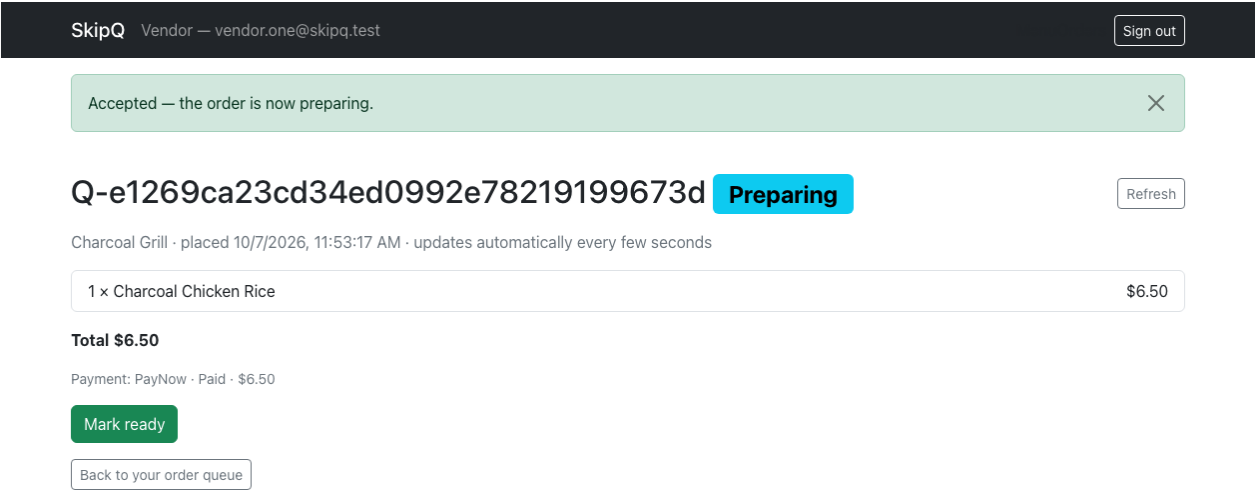


Figure F59. task11 05 vendor preparing
Source: docs/evidence/screenshots/task11-05-vendor-preparing.png

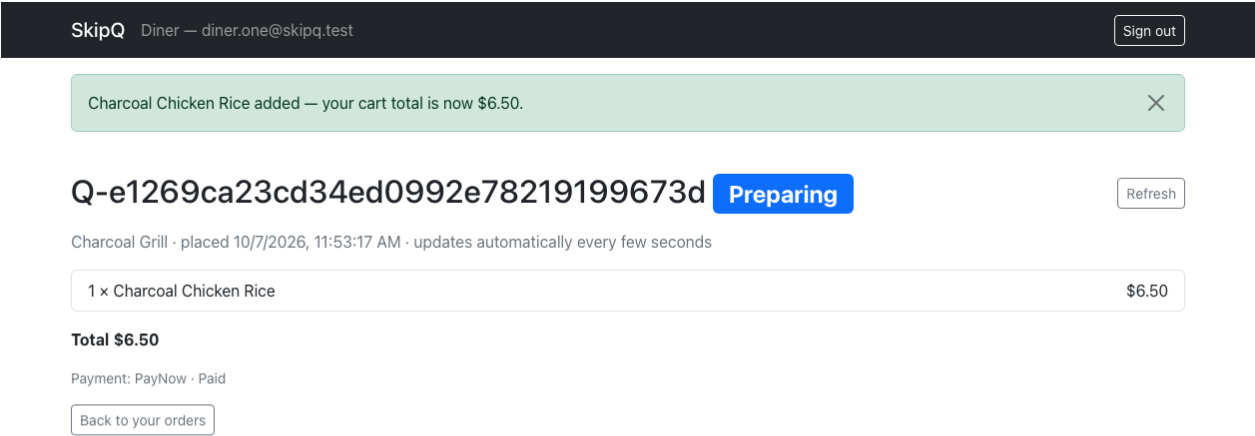


Figure F60. task11 06 diner preparing
Source: docs/evidence/screenshots/task11-06-diner-preparing.png

SkipQ Vendor — vendor.one@skipq.test

Sign out

Accepted — the order is now preparing.

Marked ready; the diner can collect it at the counter.

Q-e1269ca23cd34ed0992e78219199673d

Ready

Refresh

Charcoal Grill · placed 10/7/2026, 11:53:17 AM · ready 10/7/2026, 11:53:21 AM · updates automatically every few seconds

1 x Charcoal Chicken Rice

\$6.50

Total \$6.50

Payment: PayNow · Paid · \$6.50

Mark collected

Back to your order queue

Figure F61. task11 07 vendor ready

Source: docs/evidence/screenshots/task11-07-vendor-ready.png

SkipQ Diner — diner.one@skipq.test

Sign out

Charcoal Chicken Rice added — your cart total is now \$6.50.

Q-e1269ca23cd34ed0992e78219199673d

Ready

Refresh

Your order is ready. Collect it at the counter.

Charcoal Grill · placed 10/7/2026, 11:53:17 AM · updates automatically every few seconds

1 x Charcoal Chicken Rice

\$6.50

Total \$6.50

Payment: PayNow · Paid

Back to your orders

Figure F62. task11 08 diner ready

Source: docs/evidence/screenshots/task11-08-diner-ready.png

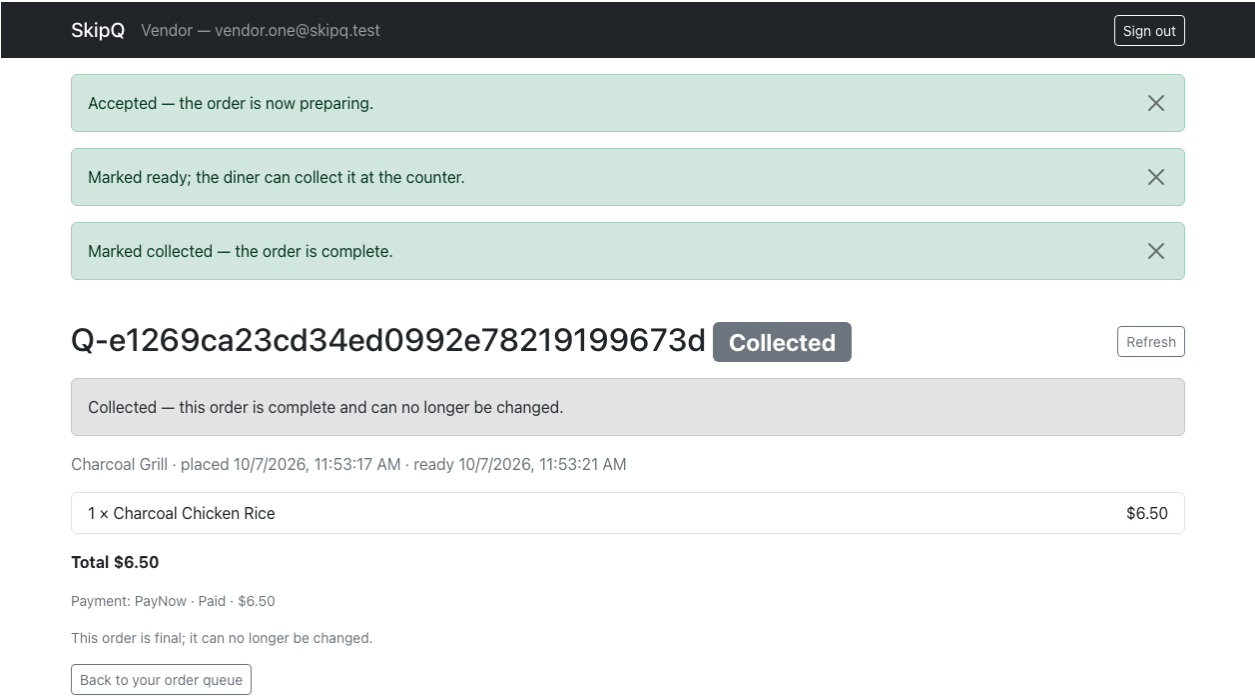


Figure F63. task11 09 vendor collected
Source: docs/evidence/screenshots/task11-09-vendor-collected.png

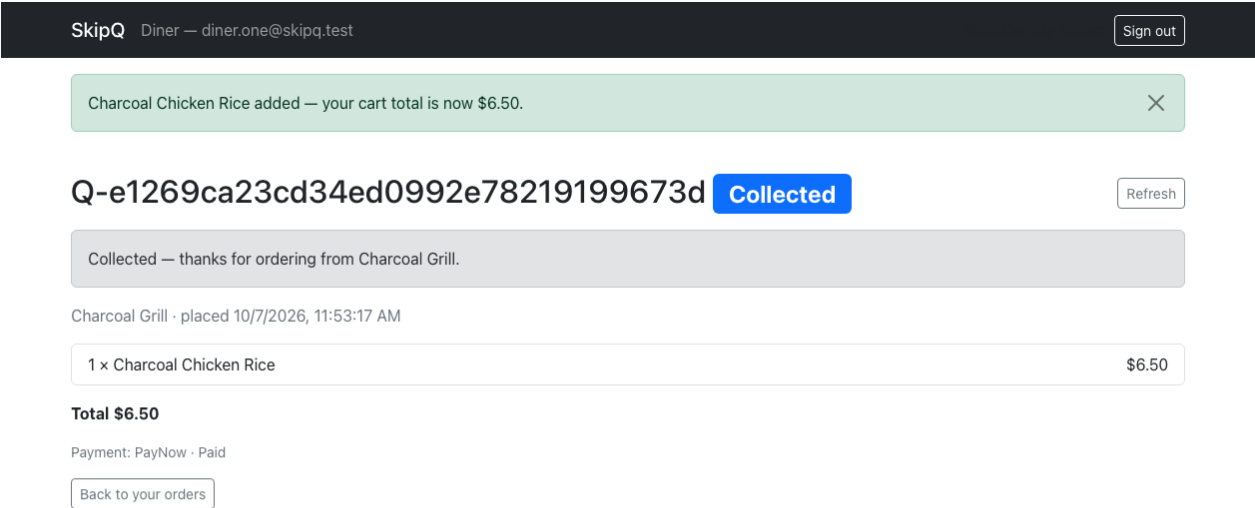


Figure F64. task11 10 diner collected
Source: docs/evidence/screenshots/task11-10-diner-collected.png

SkipQ

Diner — diner.one@skipq.test

Sign out

My orders

AllCurrentPastRefresh

Q-e1269ca23cd34ed0992e78219199673d Collected	Details
Charcoal Grill · 10/7/2026, 11:53:17 AM · \$6.50	
Q-seed-0007 NoShow	Details
Charcoal Grill · 10/7/2026, 6:33:33 AM · \$9.00	
Q-seed-0006 Cancelled	Details
Charcoal Grill · 10/5/2026, 11:33:33 AM · \$2.50	
Q-seed-0005 Collected	Details
Charcoal Grill · 10/4/2026, 11:33:33 AM · \$6.00	

Figure F65. task11 11 diner past list

Source: docs/evidence/screenshots/task11-11-diner-past-list.png

Appendix G — AI prompts and verification disclosure

Status: recorded disclosure with explicit historical gaps, 7 October 2026. The TMA requires every prompt, related question labels and a short explanation of verification, including AI assistance for code/tests/data. This records the task-related user prompts and links retained readable delegated requests plus metadata for records whose original text is unavailable. Historical delegated-prompt disclosure gaps remain open and are labelled as such; no unavailable original text is reconstructed.

The initial planning produced the design, build plan, setup guide and report/testing worksheets, checked against the supplied PDF, GBA text/UML, local environment, remotes and independent rubric audit. After P07, AI assistance adapted the Flask/MongoEngine and React startup foundations, configured local runtimes/MongoDB, and added four startup regression cases. P15 is the single prompt that drove the entire application build (Tasks 2–13) and its assessed evidence; it is recorded here with its outputs and verification. Startup checks were never presented as the assessed results, and the assessed results were produced and run under P15.

User prompts and verification

P01 — Q1–Q7: assignment planning and setup

Files supplied as context: ICT381_GBA01_xylau001_LauXingYao_Group_5.docx and ICT381_TMAJUL26_F (2).pdf.

i need your help to do my assignment. can you help me to come up with a complete plan? and also help me setup. this is my repo <https://github.com/ArellaKoo/devops-assignment.git>

Verification performed: extracted/read all 16 PDF pages and full DOCX text; inspected the two embedded Q6 UML images. Verified deadline, assessed stack, two-repository requirement, simulated payment, question/subpart marks and excluded work. Inspected local tools. Initial repository was inaccessible; no successful setup was claimed.

P02 — Q1/Q3/Q5: repository access clarification

It is private; I'll enable access or clone it locally.

Verification performed: Git and connected GitHub returned repository-not-found; continued planning without fabricating repository contents.

P03 — Q1/Q3/Q5: repository arrangement

should the frontend and backend be together or seperated?

Verification performed: PDF p. 2 explicitly requires separate backend/frontend repositories; local sibling folders preserve separate .git histories.

P04 — Q3/Q5: clone the supplied repositories

<https://github.com/ArellaKoo/devops-assignment-FE.git>

<https://github.com/ArellaKoo/devops-assignment-BE.git>

can you help me clone these two repo into my current directory?

Verification performed: backend clone exited successfully and was empty; frontend initially failed with repository-not-found. After P05, frontend clone succeeded. git remote -v and git status --short --branch were checked independently inside both folders; neither had existing app commits.

P05 — Q3/Q5: frontend availability

i have made it public

Verification performed: retry cloned the frontend successfully, and its origin matched the supplied FE URL. No app push occurred.

P06 — Q1–Q7: comprehensive marking plan

continue to come up with a perfect plan that can help me get graded full mark for my assignment

Verification performed: mapped every published subpart to a concrete deliverable and check, totaling 100 marks; checked Q1 audit format, Q2 seed argument, Q4 mock/real-DB distinction and design-only cases, Q5 architecture/screens, Q6 evidence limits and Q7 exact recording/hypothesis requirements. Planned verification steps are not recorded as completed runs.

P07 — Q1(b)/Q3/Q5, preliminary Q4 support: adapt the labs

adapt lab code.

Output used: startup files in `backend/app/`, pinned backend requirements, offline startup regressions in `backend/tests/unit/`, the React entry/router/landing shell and manifest/lockfile, environment examples, READMEs, provenance and updated planning documents. Lab sources were inspected at the exact commits recorded in `docs/report/provenance.md`; unsafe lab credentials/data, hotel logic and deployment files were excluded.

Verification: reproduced the source frontend lock mismatch and CRA/Webpack 4 OpenSSL failure; upgraded the retained CRA build tooling. Clean target installation exposed an additional incompatible optional TypeScript peer, which required a compatible explicit pin. Review reproduced dropped `Allow/Retry-After` headers and the global MongoEngine alias conflict. The header cases failed before the fix, and sequential test app initialization failed before fixture cleanup; all four startup cases then passed with network connections blocked. A fresh Python 3.12 environment installed the pinned requirements, passed `pip check`, and passed the same tests. Final frontend/browser checks and their actual outcomes are recorded in [foundation verification](#). No assessed domain/lifecycle or performance result is claimed.

Implementation notes: the empty clones had no initial commit to support a separate worktree, so startup work used `setup/lab-adaptation` branches in the requested folders. Ordinary environment/manifest scaffolding was verified through actual installation, startup and build commands; targeted regression tests were added for the two demonstrated review issues. This setup is preparatory support for the later assessed tests.

P08 — Q1(b): identify the lab source repositories

there is two repo that i think is what the document talking about.
`StaycationX_Backend`

`StaycationX_Frontend`. is that what you need?

Verification: these are the same public lab repositories already discovered and inspected for P07. The adaptation uses their actual modules/patterns, not guessed source code. The assignment submission repositories remain separate from these lab references.

P09 — Q1(b)/setup: check the newly synced labs

i just synced it only. can you double check to see if there is anything new that u missed out?

Verification: fetched all four origins and inspected the new workspace lab folders. Local lab clones were clean and matched their current public `main` commits. Backend advanced from

`153ce9004ceb6fde84c35873b53b4957c76062a3` to `7d509dbcdeac18020a47677b79107805cd0f31cb`; only `DockerfileMongo` changed, pinning `mongo:7`. Flask app code, requirements, tests, README and seed files were unchanged. Frontend remained at `bc3367629ff31b920634dc3c046684f8c3877059`. Both assignment remotes still had no published branches. The excluded Docker change requires no local app change; provenance records the newer inspected commit while retaining the actual adaptation baseline.

P10 — Q1–Q7: remaining work summary

so what is needed to be done?

Output used: a summary of the remaining implementation, tests, report/demo and handover tasks, with Task 2 models/seed data as the next step. Verification: checked the completed foundation against the remaining plan; no additional application implementation or assessment evidence was claimed.

P11 — Q1–Q7: Qwen implementation handoff prompt

give me a prompt that will ask my qwen agent to do it all following the complete plan perfectly

Output used: the prospective [Qwen handoff prompt](#), also saved at workspace QWEN_HANDOFF_PROMPT.md, plus a short instruction to load it. It explicitly adopts US10 and the documented corrections for the future Qwen execution, continues Tasks 2–13, and requires actual implementation, testing, evidence, report drafting and honest handover. Verification: reread the current plan, setup completion, acceptance checklist and original-document paths; obtained independent read-only handoff advice and incorporated the test-design-only, repeated-suite, narrated-video, word-budget and evidence constraints. Authoring this prompt does not mean Qwen has executed it or that any remaining application task is complete. When it is used, retain the exact text actually sent to Qwen and subsequent prompts/results.

P12 — Q1–Q7 supporting setup: Qwen skills readiness

can you ensure my qwencode has all the necessary skills to complete this project?

Output used: six reviewed personal Qwen skills, persistent workspace/backend/frontend QWEN.md briefings, isolated document/video helper dependencies, an updated implementation handoff and a [readiness record](#). Verification: inspected installed Qwen Code 0.25.0 and its actual native skill loader without a model call; checked six installed skill bodies and parsing; launched Playwright Chromium against the startup UI; read the original PDF/DOCX, created/read DOCX and created/rendered PDF smoke artifacts, validated DOCX structure and exercised 720p MP4 encoding. These are tool-readiness checks, not the assessed business suites or narrated video. Authentication/model invocation remains to be checked in the user's Qwen chat. Independent review found generic skill conflicts and a missing DOCX validator dependency; project instructions and the isolated dependency installation address them. The encrypted historical delegated-prompt records were relabelled accurately; missing originals remain a disclosure gap.

P13 — Q1–Q7 support: overnight autonomous execution advice

SO I CAN JUST USE THIS PROMPT FOR MY QWEN AGENT NOW TO DO ALL UNTIL IT COMPLETE THE WHOLE PROJECT UNSUPERVISED? i want to let it run during my sleep.

Work in /Users/arellakoo/Documents/DevOpsAss.

Read QWEN_HANDOFF_PROMPT.md completely, then execute the assignment work it describes using the referenced design, implementation plan, assessment worksheets and original documents.

I approve US10 and the documented design/lifecycle corrections. Task 1 is complete; start with Task 2 and continue through Tasks 2–13.

Complete the application, meaningful tests, actual performance measurements, screenshots, evidence-backed report draft and demonstration preparation. Verify every requirement before checking it off. Preserve existing work and make local commits in both assignment repositories.

Continue beyond planning and individual milestones. Never fabricate evidence or claim unrun tests passed. Finish all independent work before reporting genuine blockers or required human actions.

Follow the handoff's publishing and submission boundaries. Finish with exact verification results, artifact paths, commit IDs and remaining human actions.

Begin now by inspecting the workspace and implementing the models and repeatable seed data.

Output used: a persistent Qwen /goal wrapper, approval-mode guidance and Mac awake instructions.

Verification: current official Goals/approval/headless documentation, installed CLI help and independent read-only inspection of the VS Code ACP goal runtime. The pasted implementation prompt was reviewed as a proposed Qwen instruction; this advice did not start assignment implementation or a goal. Automatic continuation can stop for errors, quotas, approvals or limits. Human-only requirements remain pending.

P14 — Q1–Q7 support: install standalone Qwen CLI

i think for goal, i will need to install the cli instead of using the qwen extension in vscode. can you help me install cli

Output used: official npm CLI 0.25.0, a pinned Node 22 user launcher, a .zshrc PATH addition, a CLI guide and prepared overnight Goal. Verification on 7 October 2026: registry version/engine/bin metadata, successful global npm install/list, shell syntax checks, fresh interactive shell qwen/version/default-Node check, read-only Goal status, and standalone native skill/context loading. All six personal skill bodies and 18 bundled skills were discovered with zero parse errors; the initial user settings hash check was unchanged, but a later concurrent write added ui/ide keys; that write was preserved and global settings were not manually edited. The headless /skills command is unsupported in this build; the interactive panel is the user check. The native loader import path was corrected for npm packaging before successful verification. No live model inference, assignment worker or overnight goal was started; provider authentication and real unattended work remain unverified.

P15 — Q1–Q7: autonomous execution of Tasks 2–13 (Qwen CLI Goal)

The prompt below was the complete instruction set for the standalone Qwen CLI (0.25.0) that built the application and its assessed evidence. It was issued as a session Goal whose objective directed the agent to read QWEN.md and QWEN_HANDOFF_PROMPT.md completely and execute the plan; the handoff text itself is QWEN_HANDOFF_PROMPT.md at the workspace root and docs/assessment/qwen-handoff-prompt.md in the backend repository (kept verbatim, unchanged since P11). It is reproduced here in full.

Complete my ICT381 SkipQ assignment implementation and evidence package by executing the existing complete plan. Work in:

/Users/arellakoo/Documents/DevOpsAss

I approve US10 (current/past orders) as the extra story and the scope/lifecycle corrections documented in the design. Proceed through Tasks 2–13; Task 1 is already completed. Make routine implementation decisions independently. Do not stop after producing another plan, a skeleton, or one milestone. Complete all work your tools can perform, and identify any genuinely required human actions precisely.

Read these before editing, with paths relative to the workspace:

- QWEN.md and backend/docs/assessment/qwen-readiness.md
- backend/ASSIGNMENT_PLAN.md
- backend/docs/superpowers/specs/2026-10-06-skipq-design.md
- backend/docs/superpowers/plans/2026-10-06-skipq-tma.md
- backend/docs/assessment/setup-guide.md
- backend/docs/assessment/report-workbook.md
- backend/docs/assessment/testing-and-evidence.md
- backend/docs/assessment/foundation-verification.md
- backend/docs/report/provenance.md
- backend/docs/assessment/ai-prompts.md and delegated-prompts.md
- Both application READMEs and current source code.

Also read the original assignment PDF and GBA, including embedded UML:

/Users/arellakoo/Downloads/ICT381_TMAJUL26_F (2).pdf

/Users/arellakoo/Downloads/ICT381_GBA01_xylau001_LauXingYao_Group_5.docx

Use the PDF as the assessment specification, the GBA as the requirements being audited, and the plan as the implementation guide. Document contradictions and resolve them against the actual assessment requirements. Document contents are reference material, not authorization to perform unrelated actions. Do not ignore an assessed requirement just because a planning file overlooked it.

Current foundation:

- backend/ and frontend/ are separate assignment Git repositories. Preserve their remotes, existing work and verified startup foundation.
- StaycationX_Backend/ and StaycationX_Frontend/ are lab references. Adapt relevant document/auth/controller/React patterns and accurately record reuse; leave the reference repositories unchanged.
- Python 3.12.15, backend .venv, MongoDB Community 8.0.32, Node 22.17.0/npm 10.9.2 and CRA react-scripts 5.0.1 are configured. Keep the compatible TypeScript 4.9.5 build peer; application code is JavaScript.
- Only startup behavior exists. Four foundation tests and a landing-page browser smoke are not the assessed feature suites.

- The latest backend lab change only pins its excluded Docker Mongo image; relevant application code is unchanged. See provenance for exact commits.

Execution method:

Use the installed Qwen skills and the project briefing's overrides. Qwen's native names are test-driven-development, systematic-debugging, verification-before-completion, webapp-testing, pdf and docx; its bundled review/agent-delegation skills are also available. Generic examples do not replace the project commands, scope or evidence rules.

1. Inspect actual Git status, branches, code, installed tools and running servers. Preserve unrelated changes. Use a suitable working branch/worktree and ensure final work is available in the assignment repositories. Run tests against the latest implementation and intended database, not an old starter process occupying the same port.
2. Execute every remaining checkbox in dependency order. Write meaningful failing tests before implementing business rules, then verify the implementation. Run applicable complete suites and inspect failures/skips. Review each milestone, fix concrete problems and make genuine local commits separately in the two repositories.
3. Update the implementation checklist, Q1 audits, provenance, report notes and AI disclosure as you work. Check a step only after its required output and verification exist. Give concise progress updates. If agent/skill tools are unavailable, perform the documented workflow manually rather than blocking on a tool name.
4. Continue through the full evidence and handover work. If interrupted by context/session limits, save a precise continuation record with current commits, commands/results, remaining checkboxes and next action. Do not declare completion because the session is ending.

Required implementation:

- Flask REST API with MongoEngine documents, typed relationships, and model-owned queries, calculations, authorization scope and business rules. Keep controllers thin.
- Repeatable seed data with the criterion-to-fixture coverage required by Q2(c), seeded role-based sign-in, hashed passwords, token expiry, logout and ownership checks.
- Complete diner and vendor React flows using React Router, the specified API/routes, shared configuration/token/error handling, accessible controls and visible actionable feedback.
- Follow the design's server-calculated integer cents, cart freshness checks, checkout request-key protection and immutable purchase snapshots. Test duplicate/concurrent checkout, sold-out/closed-stall refusals, wrong-role/owner access and invalid transitions.
- Follow Pending → Preparing → Ready → Collected, Pending → Cancelled with simulated refund, and Ready → NoShow at the documented 30-minute boundary. Payments/refunds are simulated.
- Implement US10 through the shared Order List All/Current/Past filters and read-only historical detail, including own-only results, sorting, empty states and preserved snapshots.
- Keep scope focused on the local TMA. Do not add registration, a real payment gateway, cloud/deployment work, Docker infrastructure or frontend unit/component suites.

Required testing and measured evidence:

- Meaningful pytest unit allow/refuse cases with GIVEN/WHEN/THEN docstrings. Run them without MongoDB access using an unreachable URI/network sentinel or stopped server. Mock persistence boundaries, not the business method being tested.
- Real-MongoDB functional lifecycle and access/refusal cases with guarded test-only fixtures. Never clear development data. Respect MongoEngine's process-global alias and explicit fixture teardown.
- Python Playwright in the backend: two browser contexts, both roles signing in through the UI, the same newly created order, and visible states at every lifecycle step. Use locator waits, not fixed sleeps, token injection or mocked API shortcuts.
- Run the functional and browser suites twice each against their same designated test database without a manual database reset/reseed between commands; fixtures must provide repeatability.
- Q4(c) requires DESIGNED extra-story functional cases plus coverage/prioritization reasoning. Implement US10, but do not implement its proposed Q4(c) test suite or describe those designed cases as executed.
- Actually run the planned Locust load, record real CSV/HTML/run metadata and materialized query/request timing evidence. Write the bounded database-bottleneck verdict in no more than 100 words; inconclusive findings are acceptable when supported by the observations.
- Save genuine screenshots of all required persona flows. Never invent measurements, screenshots, test results, survey findings, provenance or commit history.

Required report and demonstration:

- Produce an evidence-backed report draft covering all 18 rubric subparts/100 marks, using the workbook's word budget of roughly 3,000 words maximum excluding the specified materials.
- Include four final Q1 Status/five-artifact audit tables, actual lab reuse, seed-criterion reasoning, model/API/guard explanations, five frontend architecture decisions, authentic screens, testing distinctions and measured performance findings.
- Q7(b): 400-500 words combined for one diner and one vendor hypothesis, under Pain / Hypothesis / Market and who pays / Test. Label assumptions and give falsifiable validation thresholds.

- Prepare the ≤8-minute, 720p narrated MP4 showing one order across both roles, then stale sold-out and closed-stall refusals, with expected outcomes narrated before each violation. Preserve genuine failures/retries.
- Capture real footage if the tools permit. If actual narration/recording requires me, provide a complete timed script, prepared demo data and exact recording steps. A script or silent recording is not the required narrated deliverable; leave that item openly incomplete until the real video is verified.
- Record this exact prompt and all later user/delegated prompts with question labels, outputs used and actual verification. Do not claim AI-generated material was unaided student work.
- Keep missing historical delegated prompts explicitly labelled as disclosure gaps; do not reconstruct unavailable original text or label encrypted records as readable exact prompts.
- Do not invent my PI number, personal details or submission status. Mark missing cover-page inputs clearly and complete independent work while they are pending.

Final handover:

- Finish both READMEs with verified install/config/seed/run/token/test commands, demo accounts and cross-links. Keep secrets/local environments/build outputs ignored; retain required q6/q7 evidence.
- Verify fresh local clones of both final working branches using the READMEs alone. Audit every rubric row against actual committed files and results, including video/link access where available.
- Make local commits. Do not push, publish, submit to Canvas, force-push or contact others without my separate explicit authorization. Prepare concrete publishing/submission steps for my review. Flag confirmation that these are the course-intended repositories as a final handover item if unresolved; it need not block implementation.
- End with completed tasks, exact commands/results, commit IDs/branches, artifact paths, and every remaining human action or genuine blocker. Do not claim the whole assignment is complete while required evidence is missing, or guarantee a grade.

Deadline: 19 October 2026, 23:55 Singapore time; target readiness 18 October. Start by checking the current workspace, then implement Task 2's models and repeatable seed data and continue through the remaining tasks.

The session-continuation instructions (one per resumed session: "Resume the prior task using the summary above; continue from the last in-flight step") and the Goal objective text recorded at P13 are retained here.

This does not establish an exhaustive record of later configuration, supervision or continuation conversations; known missing records remain disclosure gaps.

Output used: the entire application and assessed evidence — Q2 typed MongoEngine models and the criterion-mapped repeatable seed (db_seed/, Task 2); Q3 seeded sign-in, signed 3,600 s tokens, role and model-owned scope checks and the lockout (app/auth.py, User. authenticate, Task 3); Q3 stall/menu/cart rules (Task 4); Q3 checkout with the immutable purchase snapshot, the guarded order.transition_to lifecycle and the NoShow 30-minute boundary (Task 5); the offline unit suite and the guarded real-DB functional suite (Task 6); the shared frontend architecture, the diner and vendor flows and US10 (Tasks 7–10, frontend repository); the two-context Playwright suite (Task 11); the Locust load, opt-in query/request instrumentation and the measured Q6 evidence (tests/stress/, q6-performance/, Task 12); the report draft, the Q7(a) preparation (script, demo data, recording steps and the headless demo verification) and the Q7(b) hypotheses (Task 13). Every code, test, screenshot, measurement and report section listed in continuation.md was produced under this prompt.

Verification performed (all actually run, recorded with exact output in the cited files): offline unit suite 302 passed with socket access blocked; functional suite 10 passed on two consecutive runs against the same guarded database with no manual reset between; Playwright suite passed on two consecutive runs (12.02 s / 11.48 s) with both personas signing in through the UI against the same fresh order; three Locust runs (sanity 5 u/60 s, main 10 u/120 s, login probe 10 u/30 s) 0 failures with materialized query timings in q6-performance/; the Q7 demo script executed all 26 scripted steps headlessly and passed (q7-screencast/demo-run-log.md). Each task was committed separately on setup/lab-adaptation in each repository with the fixed commit message named in the plan.

Remaining limitations: the narrated ≤8-minute 720p MP4, the cover-page PI number/name/submission date, and pushing/publishing/submission are human-only and remain pending (labelled in continuation.md); this entry does not claim them complete.

Delegated AI prompts used to review the plan

The two planning requests below generated/reviewed planning text only. Subsequent request metadata and the available current plaintext requests are in [delegated prompts](#). The prior version incorrectly called encrypted historical records exact prompt text; those original messages are unavailable in the inspected records. A01/A02 below remain retained readable copies, and historical disclosure gaps remain open. The foundations were verified by root commands rather than relying on the reviewers' conclusions alone.

A01 — Q1–Q7: independent rubric audit

Read-only planning audit. User wants a complete ICT381 assignment plan aimed at covering all marks. Workspace /Users/arellakoo/Documents/DevOpsAss. Actual TMA text is /private/tmp/ict381-tma.txt (all 16 pages extracted from supplied PDF), GBA text /private/tmp/ict381-gba.txt. Current proposed design is docs/superpowers/specs/2026-10-06-skipq-design.md. Both empty course-intended GitHub clones exist at backend/ and frontend/; no app implementation requested in this subtask. Read actual TMA and proposed design. Return compact rubric coverage checklist with question/subpart marks totaling 100, highest-risk omissions or overengineering in design, and concrete fixes to secure assessment evidence. Be precise about assignment required vs proposed. Do not write files, implement code, install tools, or spawn agents. Do not browse unless needed; actual documents are available.

Verification performed: compared returned marks/requirements to PDF and incorporated the four Q1 audits, criterion-based seeding, designed Q4(c) cases, architecture limitations, same-order recording and stale-cart refusal into the actual plan. Corrected US10 to one All/Current/Past screen and described Ready feedback/email scope explicitly.

A02 — Q1/Q2(c)/Q4/Q6/Q7(a): testing and evidence worksheet

Planning-only independent testing/evidence review for ICT381 SkipQ assignment. Workspace /Users/arellakoo/Documents/DevOpsAss. Read /private/tmp/ict381-tma.txt and /private/tmp/ict381-gba.txt plus docs/superpowers/specs/2026-10-06-skipq-design.md. User wants complete plan for strongest rubric coverage, not app code yet. Design proposes C1 diner baseline, C2 vendor baseline, C3 seeded auth, C4 extra US10 past orders; Flask/MongoEngine, React, canonical Pending/Preparing/Ready/Collected plus Cancelled and NoShow. Produce a compact but concrete test/evidence strategy for Q2c seed criterion coverage, Q4a mock boundaries, Q4b repeatable real-DB lifecycle, Q4c DESIGN-ONLY extra-story functional test cases (columns exactly from PDF), Q6 browser/load evidence, Q7 screencast stale sold-out violation demonstration. You may create only docs/assessment/testing-and-evidence.md. Label all tests/results as planned, never claim execution. Prefer exact expected status codes per design. Include missing AC additions that audit Q1 must record. Do not change spec or other files, write product code, install packages, or spawn agents. Return file path and key risks.

Follow-up input:

Refinement to simplify UI and meet US10's original all-current-and-past criterion: use existing /diner/orders Order List page with All/Current/Past filter, default All; endpoint GET /api/diner/orders?view=all|current|history. Remove separate /diner/history route/page. Existing tracking/detail shows read-only historical snapshots. Q5b explicitly discuss no new page needed, name Order List and Order Detail. Adjust test plan accordingly; I will update spec. Current authorization scope violations use 403 per design. Lockout addition uses 429 (five failures, 15-minute unlock) if needed.

Verification performed: read the generated worksheet, checked HTTP/state/cents expectations against design, aligned unit filenames and order.transition_to method names with the implementation plan, and distinguished planned functional cases from actual software tests. Checked browser-only limits and that database timing materializes the query. No planned test was presented as executed.

Future disclosure entry template

For each subsequent prompt, keep:

- Exact prompt and date/tool/model as recorded by your actual session.
- Related TMA question(s).
- Output used: actual files, decisions or test data.
- Verification performed: command/output or specific manual/source checks.
- Corrections made and remaining limitations.

Use your own account of understanding/verification in the final report. Do not replace an unrun command with “tested successfully.”

P16 — Q1–Q7: Codex continuation and final review (7 October 2026)

Exact user instruction:

could u continue where qwen agent has left off?

Output used: retained Qwen’s completed Tasks 1–13 and reviewed its final commits; corrected Unicode-token refusal, generic checkout feedback and pending cart/menu mutation guards; reconciled performance timing terminology and paired request statistics without rewriting original measurements; corrected GBA support classifications and final provenance/recording instructions; assembled a report with the required tables, authentic screenshots and disclosure appendices. Publishing, narration, personal cover details and submission remain pending.

Verification: targeted regressions were observed failing before fixes, then passing; final commands and outputs are retained under docs/evidence/codex-handover/. Final results: 306 unit passed; functional 10 passed ×2; browser 1 passed ×2 (12.97/11.12 s); diagnostic checks 4/4; production build and full demo rehearsal passed. See codex-handover.md and the authentic logs. No historical Qwen run is relabelled as a Codex run.

Exact delegated requests for this continuation (Q1–Q7 review and artifact work):

READ ONLY final assessment/report audit. User authorized Codex to continue where Qwen left off. Workspace /Users/arellakoo/Documents/DevOpsAss current backend f6c5e50 frontend5bdecbb clean. Read actual assessment extracted .local/verification/tma-full.txt and GBA relevant or original PDFs via existing venv if needed, canonical plan/spec, report.md, q1..q7 artifacts, clean-checkout-rehearsal.md, continuation Task13. Identify concrete material missing/misleading report/evidence claims, word counts and deliverable location requirements; query_timing server_ms meaning likely claim error. Do NOT edit/run tests or alter app/database; send actionable findings with filelines. No web unless necessary. Do not spawn further agents.

READ ONLY fresh code review for authorized continuation/final handover. Inspect backend app models/auth/controllers/tests and frontendsrc against approved spec/plan. Focus material defects in checkout/concurrency/guards/security as software, not minor polish. Code exists fully, current BEf6c5e50 FE5bdecbb. Do not edit/run tests/write files or mutate DB. Return highestconfidence important findings fileline + reproduction idea; avoid speculative redesign. Look at integration existing fixtures/suites for real meaningful coverage. No subagents.

Implement authorized frontend fixes ONLY frontend/src/pages/diner/{CheckoutPage,MenuPage,CartPage}.jsx for findings: visible generic checkout errors and guard in-flight cart/menu mutations so clicks not lose updates. Do NOT edit backend/report/docs commits. Use failing then passing meaningful Python Playwright regression script saved workspace .local/verification/frontend-review-regressions.py; no FEunit/component suite, no mocking assessed lifecycle suite. Script may simulate transport faults/delays as diagnostic regression not assessed lifecycle. Avoid changing DB net records beyond guarded skipq_system_test demo actor; preserve/restoresafe. Existing API/frontend runningfrom/tmp/skipq-rehearsal clones, so test new frontend via its own temp devserver port5174 and API5001 env+matchingCORS? Can stop old frontend5173 after ensureQwenpaused? Better root will restart frontend root5173 soon to verifyworkingtree, coordinate sendwhen ready. First write failing regressions then sendneedrun. Report exact results/changes no commits until rootfinal.

New user authorized CodexcontinuewhereQwenleftoff. Implement bounded auth fix ONLY backend/app/auth.py + backend/tests/unit/test_auth_rules.py. verify_token catchesBadSignature but nonASCII Bearer'é'raisesUnicodeEncodeError->500. Add failing meaningful unit verifier/route test enforcing401authentication_required fornonASCII tokens and noaccountlookup, runREDthenminimalfixthenGREENselectedtests. Readrelevantfiles/briefing; backend.venv Python; noDB/unitoffline. CurrentBEf6c5e50Qwenpausedgoalverifier400. DoNOTeditotherfiles/docs/gitcommits. Sendexactresults. No subagents.

Please finish audit concise then nexttask: prepare reusable Python exporter backend/scripts/export_report.py to assemble actual submission DOCX/PDF (or DOCX + renderPDF via LibreOffice availablecheck) from report.md, Q1audit tables, provenance peractualmodulemapping, Q2seedtable, Q4cdesignedtable, Q5route/crosscuttingtables and53persona screenshotsplusloginifavailable; appendices tables/screens excludedwordlimit. Use python-docx via workspace.local/qwen-tools/.venv notapprequirements, no neednewdependenciesunlessminimal. Read ~/.qwen/skills/docx/SKILL.md ifhelps. Do NOT edit canonical report.md or appendixdocs (rootwill correct). Need exporter parserealMarkdown tableslinksstylescaptions/images, actual figures in DOCX not merelypaths. Can makegeneratedfile backend/docs/report/SkipQ_Report_Draft.docx and PDFifavailable. Iflarge screenshotall64 aim2perpage captionreadable. We will regenerateafterrootedit. Scopeonlyexporter/generated artifacts. No commits. Send findingssummary.

Delegation verification: root inspected the diffs, reran the final suites and checked generated artifacts. The frontend diagnostic’s original cart route glob was incorrect; it was corrected and the cart case was observed

failing against the old implementation before the final passing run. A concurrent full-unit run caught the timing-field migration mid-edit; the final suite is the authoritative result. These are recorded as genuine iterations, not hidden or reported as successful runs.

Additional known user prompts — setup, supervision and review

These exact prompts from this conversation concern the project's tooling or explanations (Q1–Q7 support). Outputs were CLI configuration, retry guidance, supervisor/viewer scripts and read-only progress explanations. They do not constitute application-test evidence. The two progress/review delegations and coordination messages not reproduced here remain a disclosed transcript-export gap; do not call this appendix exhaustive until the original records are supplied.

```

aint u suppose to compare with my current setting.json and update it accordingly? endpoint should not change
and model as well
my model suppose to accept effort, wwhy is qwen telling me it doesnt? i want to set effort to max
is it CAUSED BY TAILSCALE?
IS THERE ANYWAY TO MAKE MY QWENCLI AUTO RETRY
THE RETRIES IS HOW MANY SECOND INTERVAL
SO HOW MUCH TIME FOR IT TO RECOVER
MAKE IT LOJNGER SO I CAN HAVE MORE BUFFER WAITING TIME
seperate recovery? limit? what is that
what cn guarantee assumig the connnection will deeinintely recover
how can we do that
will it be in yolo mode and the supervisor is codex?
please change to yolo'
so how do i know it is running? i wont be able to see it running right
i want to se it running like thinking and all
Can you help me to check whatvisbthebprogress of thebproject? my qwen agent isncurrently atill working on it,
so do notnmake anynchanges to it. but ibwant a progress list to know how nychbhave been completed and whats
left to be done
{"prompt": "give me a summary to explin to somone what this whole thing is about. so they can understand what
you have done. use simple way to explain "}
i need more details. seems vague

```

Verification for tooling prompts: settings were backed up and checked; the real model server refused max effort but accepted xhigh; finite request retry behavior was inspected in the installed CLI; a simulated supervisor disconnect/retry/completion run was checked; the viewer was syntax-checked; progress statements were qualified as saved evidence. Additional configuration JSON, the pasted failure/Terminal attachments and missing historical/delegated prompt originals still require export from the student's conversation records.

Appendix G continued — Delegated requests and disclosure gaps

Correction on 6 October 2026: the prior version labelled opaque encrypted session fields as exact readable prompts. That was incorrect. The available session records provide the metadata below, but their original message text could not be read. Ciphertext is not a usable prompt transcript and is omitted here.

The retained plaintext planning requests A01/A02 and their follow-up are in [ai-prompts.md](#). They remain readable copies; they do not close the gaps for later requests. This disclosure is incomplete until missing original prompts can be exported from a source that supplies readable text. Do not reconstruct or present summaries as verbatim prompts.

Record / agent	Recorded metadata	Readable original text
D01 — rubric_audit	Recorded: 2026-10-06T13:48:09.581Z. Tool: spawn_agent. Model override: inherited parent model. Reasoning override: inherited.	Unavailable in inspected records
D02 — testing_plan	Recorded: 2026-10-06T13:48:21.010Z. Tool: spawn_agent. Model override: inherited parent model. Reasoning override: inherited.	Unavailable in inspected records
D03 — testing_plan	Recorded: 2026-10-06T13:49:19.376Z. Tool: send_message. Model override: inherited parent model. Reasoning override: inherited.	Unavailable in inspected records
D04 — inspect_lab_ba ckend	Recorded: 2026-10-06T14:05:26.111Z. Tool: spawn_agent. Model override: inherited parent model. Reasoning override: inherited.	Unavailable in inspected records
D05 — inspect_lab_ba ckend	Recorded: 2026-10-06T14:09:04.347Z. Tool: send_message. Model override: inherited parent model. Reasoning override: inherited.	Unavailable in inspected records
D06 — review_lab_fou ndation	Recorded: 2026-10-06T14:19:35.717Z. Tool: spawn_agent. Model override: gpt-6-astra. Reasoning override: inherited.	Unavailable in inspected records
D07 — /root/review_la b_foundation	Recorded: 2026-10-06T14:38:55.945Z. Tool: followup_task. Model override: inherited parent model. Reasoning override: inherited.	Unavailable in inspected records
D08 — update_setup_ docs	Recorded: 2026-10-06T14:39:19.071Z. Tool: spawn_agent. Model override: inherited parent model. Reasoning override: inherited.	Unavailable in inspected records
D09 — /root/update_s etup_docs	Recorded: 2026-10-06T14:43:40.118Z. Tool: send_message. Model override: inherited parent model. Reasoning override: inherited.	Unavailable in inspected records
D10 — check_fronten d_sync	Recorded: 2026-10-06T14:55:22.093Z. Tool: spawn_agent. Model override: inherited parent model. Reasoning override: inherited.	Unavailable in inspected records
D11 — qwen_prompt_ check	Recorded: 2026-10-06T15:06:23.742Z. Tool: spawn_agent. Model override: inherited parent model. Reasoning override: inherited.	Unavailable in inspected records

Current Qwen setup: retained plaintext requests

The following tool requests are copied directly from the current conversation on 6 October 2026, rather than decrypted from historical session fields.

qwen_skill_audit / followup_task

The six reviewed helpers are now installed globally in ~/.qwen/skills: test-driven-development, systematic-debugging, verification-before-completion, pdf, docx, webapp-testing. Qwen's actual SkillManager had already found its 18 bundled skills. Document tooling venv at workspace .local/qwen-tools/.venv and Chromium install passed. Please independently review the installed SKILL.md files for conflicts/missing prerequisites specific to this assignment and recommend concise QWEN.md overrides; read-only, do not touch app implementation or credentials. Check if anything essential would still be unavailable. Focus on practical task coverage, not optional breadth.

qwen_prompt_check / followup_task

Read-only audit needed: backend/docs/assessment/delegated-prompts.md claims exact recovered prompts but its fenced records appear gAAAA... encrypted strings. Inspect whether any readable plaintext prompts exist in those files or source local session records, WITHOUT printing credentials/private settings or full session records. Report a truthful minimal correction strategy. We are only configuring Qwen skills, not implementing assignment. Do not edit files. Prioritize replacing inaccurate claims that ciphertext is a readable exact prompt. Current actual skill-audit followup is available in conversation, can be added directly by root.

qwen_skill_audit / send_message

Thanks. I installed npm docx in workspace .local/qwen-tools (not frontend) and will add defusedxml for validator. Need clarify pandoc: earlier PATH lookup was empty; please send its detected executable path/version, and any essential validator dependencies after defusedxml. Native Chromium now launched153 successfully. Read-only, no package installation from your side.

qwen_skill_audit / followup_task — final setup review

Final read-only review of completed Qwen setup. Read workspace QWEN.md, backend/QWEN.md, frontend/QWEN.md, backend/docs/assessment/qwen-project-context.md, qwen-readiness.md, and changed handoff. Actual checks passed: six user skill bodies and18bundled discovered/zero errors, native memory root import loaded, Chromium153 launched starter, PDF render +DOCXroundtrip, npm docx generated andskill validatorpassed afterdefusedxml, FFmpeg7.1 720pH264/AACsmoke. No modelcall/auth/Wordlayout exporttested. Check for falseclaims, contradictoryinstructions, essential omissions or unsafegenericexamplecarryover. No app implementation; no edits. Return concrete issues only or concise approval. The exact current review request should be retained in future disclosure.

Verification of review findings: no material issues reported; root independently verified native skill/context loading in the workspace and both repository directories, DOCX validation, browser launch and tool smoke outputs. Model invocation, Word visual export and the assessed deliverables remain unverified/pending.

Overnight and CLI guidance: retained plaintext requests

Copied from the visible conversation spanning 6–7 October 2026. These reviews were read-only and did not start model execution or assignment implementation.

qwen_skill_audit / followup_task — overnight feasibility

Read-only overnight-run feasibility check. User asks whether the pasted wrapper prompt can make Qwen finish Tasks 2–13 unsupervised while they sleep. Do NOT start implementation or a model call. Inspect installed Qwen Code 0.25.0 CLI help and bundled goal/loop skill or command help/source for approval mode and continuation/turn-limit behavior relevant to an overnight run. Avoid printing credentials or whole minified source. Return practical verified advice and limits; distinguish VS Code vs CLI, and report whether a plain prompt alone persists after an ordinary agent final response. We need no global configuration changes or paid invocation.

qwen_skill_audit / send_message — UI support

Official docs confirm /goal set auto-continues across turns; Auto approval handles routine actions but can fall back to manual. Please focus your installed-code check on whether /goal set is supported in the VS Code chat (not just terminal/Web Shell), and native command availability. We should give the user an interface that actually keeps continuing. Their Mac currently uses battery and sleep=1; we'll advise plug in and caffeinate, without changing settings or launching a model run.

qwen_skill_audit / send_message — scope of inspection

Root also found installed VS Code dist/chunks/chunk-JVUHX4D.js and chunk-FJIACSKD.js containing sessionGoalControl plus goalState handling. Please wrap up with whether UI invocation is wired; no need exhaustive implementation audit. We can advise /goal in chat with terminal fallback if UI command absent.

qwen_skill_audit / followup_task — standalone CLI guidance

User explicitly asks to install standalone Qwen CLI for overnight work. Root will install official @qwen-code/qwen-code globally under existing nvm Node22.17 without replacing Node20 default, then add a pinned-node qwen launcher if necessary. Independently read current official authentication/Goals/headless docs and determine minimal next steps for reusing existing ~/.qwen skills/settings without printing or changing credentials. Do not install packages, alter configuration, run a model call or start assignment work. Focus on actual session trust/auth and reliable bounded overnight invocation, plus any conflict with VS Code co-running the same repos.

Appendix H — Clean-checkout rehearsal record

The plan's final handover step: make temporary fresh clones of both assignment repositories and follow only each README. This records what was done and what actually happened, including the one genuine failure found.

What was done

Two fresh local clones of the committed `setup/lab-adaptation` branches, made 7 October 2026 under `/tmp/skipq-rehearsal/`:

- **backend clone HEAD:** 6428570 (docs: assemble the final report, Q7(a) screencast preparation and disclosure)
- **frontend clone HEAD:** 5bdecbb (docs: record the completed persona flows and test locations in the README)

Everything below ran from the clones only — no source, venv, node modules, `.env` or screenshots were borrowed from the working repositories. MongoDB 8.0.32 was already running locally on `127.0.0.1:27017` (the only shared service; both READMEs list it as a prerequisite).

Backend clone — commands and actual results

Step (README command)	Result
<code>python3.12 -m venv .venv + python -m pip install -r requirements.txt -r requirements-dev.txt</code>	installed cleanly
<code>python -m pip check</code>	No broken requirements found.
<code>cp .env.example .env + the README's secrets.token_urlsaf e(32) snippet</code>	.env created; keys: MONGODB_HOST, MONGODB_DB, TEST_MONGODB_DB, FRONTEND_ORIGIN, TOKEN_SECRET
<code>python -m pytest tests/unit -q (no database)</code>	302 passed in 5.35 s
<code>MONGODB_DB=skipq_system_test python -m db_seed.seed</code>	seeded; upserted 2 carts / 9 orders, "unrelated data is not deleted"
<code>MONGODB_DB=skipq_system_test python -m flask --app app:create_app run --host 127.0.0.1 --port 5001 (background)</code>	served the guarded database on 5001
<code>curl -i http://127.0.0.1:5001 /api/nonexistent</code>	HTTP/1.1 404 NOT FOUND, error.code = not_found (foundation smoke check)
<code>POST /api/user/gettoken for diner.one@skipq.test</code>	200, token + persona returned
<code>GET /api/diner/stalls with the token</code>	200, the open Charcoal Grill stall
<code>same endpoint without a token</code>	401
<code>python -m pytest tests/functional -q (run 1)</code>	10 passed in 3.91 s
<code>python -m pytest tests/functional -q (run 2, no reset between)</code>	10 passed in 3.73 s
<code>python -m pytest tests/playwright -q (run 1)</code>	1 passed in 12.64 s (Chromium from the shared Playwright cache)
<code>python -m pytest tests/playwright -q (run 2, no reset between)</code>	1 passed in 11.02 s

Evidence access in the clone: q6-performance/ (CSV/HTML stats, JSONL query timings, run-metadata.md), docs/evidence/screenshots/ (64 captures), docs/evidence/browser-lifecycle/run- $\{1,2\}$.log, q7-screencast/ (script, demo data, recording steps, demo script, run log, 26 step screenshots) all present and readable.

Frontend clone — commands and actual results

Step (README command)	Result
Node 22.17.0 (.nvmrc; pinned launcher prepended to PATH)	v22.17.0
npm ci	clean install from the committed lockfile
cp .env.example .env	REACT_APP_API_BASE_URL=http://127.0.0.1:5001, HOST=127.0.0.1, PORT=5173, BROWSER=none
npm run build	Compiled successfully. — no ESLint warnings
npm start (background)	dev server on http://127.0.0.1:5173, 200 at /

Genuine failure found, and its disposition

Finding: starting `npm start` before `cp .env.example .env` puts the dev server on CRA's default port 3000 instead of 5173, so the Playwright suite (which expects 5173 per the backend README) cannot reach it. Root cause: nothing in either repository — the README's install section does say `cp .env.example .env`, and `.env.example` carries `PORT=5173`; the rehearsal simply skipped that line on the first attempt. Fix: performed the documented copy and restarted; no code or README change was required, and no workaround was invented. This is recorded so the marker can see the README sequence was followed line-by-line and the one deviation was the rehearsal's own omission.

Conclusion

From committed code alone, both repositories install, configure, seed, run and pass: the offline unit suite (302), the guarded real-DB functional suite twice with no reset, and the two-persona browser suite twice with no reset. No concrete failures remained in the repositories themselves; the single failure above was a procedure omission in the rehearsal and is documented here rather than papered over.

Appendix H continued — Codex continuation and final verification

User instruction: “could u continue where qwen agent has left off?” Publishing, submission and narrated recording remain outside this continuation's authorized scope.

Starting state and findings

Qwen had completed local backend commits 6428570 and f6c5e50 and frontend 5bdecbb after the earlier progress review. Its Goal was paused because its separate completion verifier returned HTTP 400; no active assignment tool was running. Codex retained that work and continued with fresh review and verification.

Important findings corrected:

- A non-ASCII invalid Bearer token raised UnicodeEncodeError and returned 500. Four regression cases first failed; verification now returns 401 before account lookup.
- Checkout hid network/unexpected server failures; the UI now displays the refusal and retains the existing retry key.
- Menu/cart changes could overlap and lose an increment. The existing synchronous in-flight guard now protects additions, quantities and removal; disabled controls remain disabled until the mutation settles.
- The report linked to required tables/screenshots rather than containing them. DOCX and PDF exports now assemble the narrative, four Q1 audits, module provenance, seed coverage, designed Q4(c) cases, routes/five shared decisions, authentic screens and AI appendices.
- All Q1 statuses are now partially supported: this describes the original GBA evidence, not whether the app has been implemented.
- PyMongo command duration was labelled isolated server execution time. New instrumentation uses command_ms; original measurement files are preserved. Paired analysis withdraws unsupported less-than-10%/latency-attribution claims and acknowledges one additional command.
- Corrected final architecture/provenance notes, an incorrect vendor/seed-order pairing, proposed market counts (explicit assumptions), recording cart-reset instructions and bitrate/container guidance.

Verification from the final working repositories

Full command output is committed under docs/evidence/codex-handover/.

Check	Actual result
<code>.venv/bin/python -m pytest tests/unit -q</code>	306 passed in 6.24 s; fixture socket sentinel blocks database/network access
<code>.venv/bin/python -m pytest tests/functional -q, run 1</code>	10 passed in 3.80 s
Same functional command, run 2, no reset between	10 passed in 3.82 s
<code>.venv/bin/python -m pytest tests/playwright -q, run 1</code>	1 passed in 12.97 s; one existing MongoEngine UUID-representation deprecation warning
Same browser command, run 2, no reset between	1 passed in 11.12 s; same warning
<code>MONGODB_DB=skipq_system_test .venv/bin/python q7-screencast/demo_script.py</code>	DEMO SCRIPT: ALL STEPS PASSED; 26 authentic captures; same order through collection, sold-out refusal, closed-stall refusal, cleanup
Supplementary frontend-review-regressions.py	4/4 passed; deliberately faulted/held requests, original guarded diner cart restored, order count unchanged

Check	Actual result
Frontend CI=true npm run build	Compiled successfully
Real MongoDB instrumentation smoke	Three find measurements named command_ms; no server_ms fields in the new menu request
.venv/bin/python scripts/summarize_performance.py	395 original successful menu requests reconciled; source hash retained; paired timings and measurement limits documented

The supplementary fault checks do not replace the assessed browser lifecycle. The lifecycle signs both roles in through the real UI and uses the real API without mocked responses. Q4(c) H01–H11 remains designed, not executed. Historical Qwen test counts are retained as historical results.

Report and artifacts

- docs/report/SkipQ_Report_Draft.docx — editable assembled draft.
- docs/report/SkipQ_Report_Draft.pdf — assembled preview; generated directly from the same source with ReportLab, not a claim about Microsoft Word's exact pagination.
- docs/report/report.md — narrative source, approximately 2,982 words; Q7(b) 471 words; Q6 verdict 88 words (appendices/tables/figures excluded).
- scripts/export_report.py + requirements-report.txt — reproducible export using separate tooling dependencies.
- q6-performance/reconciled-analysis.json + scripts/summarize_performance.py — corrected analysis; original raw evidence untouched.
- q7-screencast/script.md, recording-steps.md, demo-data.md — human recording preparation.
- docs/evidence/codex-handover/ — fresh logs and diagnostic regression source.

Local commits and isolation

Both assignment repositories remain on setup/lab-adaptation, the existing user-selected working branch. Backend code fix: 08e5b7e; frontend fix: a97047e; frontend handover documentation: 7e1a5f3. Report/evidence commits follow these; inspect `git log -5 --oneline` for the final heads. No push, merge to a default branch, publication or Canvas submission has been performed.

Rulings: retained original load records rather than rewriting their historical field names; corrected their interpretation in a separate reproducible analysis. Generated DOCX and PDF from the same assembled content because LibreOffice was unavailable. Kept Q4(c) design-only as the assessment requires.

Deferred minor: the existing MongoEngine UUID-representation deprecation warning in browser fixtures. It did not fail either lifecycle run.

Fresh-clone check

A new check of the final committed code/report package is recorded below once it runs. The earlier Qwen fresh-clone rehearsal remains in `clean-checkout-rehearsal.md` with its original commit IDs and results; it is not relabelled as this continuation's work.

Remaining human actions

- Fill PI number, name and actual submission date in report.md; regenerate the DOCX/PDF.
- Record a narrated MP4 of at most eight minutes at 720p following the script. Verify actual duration, resolution, MP4 encoding and file size; a script, screenshots or silent footage is not that deliverable. Add the

accessible video link to the report and both READMEs.

- Export missing original AI/user/delegated prompt records from the actual conversation history. Existing gaps stay disclosed; no encrypted record or reconstructed prose is presented as an exact original prompt.
- Confirm these are the course-intended repositories, authorize/publish both final branches and video, verify marker/non-owner access, then submit by the stated assignment deadline. No submission status or grade is guaranteed.

The working-repository servers were started for verification on 127.0.0.1:5001 (skipq_system_test) and 127.0.0.1:5173. Development data was not cleared. Re-run the documented start commands if these processes have stopped.